Universidad Tecnológica Nacional
Facultad Regional Bahía Blanca

Ingeniería Electrónica

Proyecto Final de Carrera

SIMULADOR DE VARIABLES DE PROCESO PARA LAZOS DE CONTROL INDUSTRIAL

Autor
Nicolas Emiliano Ivani

Director
Ing. CONDE, ALFREDO OSCAR

Co-Director
Ing. ZABALOY, MARIO ESTEBAN

Docentes de la cátedra de Proyecto Final:
Ing. LAIUPPA, ADRIÁN H.
Ing. GALASSO, CHRISTIAN L.

Bahía Blanca | 2026

Índice

1. INTRODUCCIÓN 1
   1.1. Problemática y necesidad identificada 1
   1.2. Objetivos 1
   1.3. Alcance y limitaciones 2
2. FUNDAMENTOS TEÓRICOS 3
   2.1. Normalización de las señales 3
   2.2. Modelo de primer orden con tiempo muerto 4
   2.3. Implementación discreta 5
3. ARQUITECTURA Y FUNCIONAMIENTO GENERAL 6
   3.1. Conexión con el PLC 6
   3.2. Organización interna y temporización 6
   3.3. Modos de funcionamiento y configuración 7
   3.4. Determinación de los presets 9
4. DESARROLLO DEL HARDWARE 10
   4.1. Unidad de procesamiento y alimentación 10
   4.2. Circuito de entrada de 4–20 mA 12
   4.3. Circuito de salida de 4–20 mA 14
   4.4. Interfaz local y periféricos 17
   4.5. Diseño de las placas e integración física 18
5. DESARROLLO DEL FIRMWARE 22
   5.1. Organización general 23
   5.2. Adquisición y procesamiento de la entrada 23
   5.3. Cálculo del proceso y generación de la salida 24
   5.4. Interfaces y almacenamiento de la configuración 26
   5.5. Registro de datos 26
6. INTEGRACIÓN CON EL PLC 28
   6.1. Equipamiento y conexión 28
   6.2. Programa de control 29
   6.3. Interfaz de operación 30
7. ENSAYOS Y RESULTADOS 31
   7.1. Verificación de las señales de entrada y salida 31
   7.2. Pruebas funcionales e integración con el PLC 32
   7.3. Verificación del modelo de proceso 32
   7.4. Verificación de offset, ruido y congelamiento 34
8. CONCLUSIONES 36
9. Bibliografía 37

ANEXO A – FRAGMENTOS PRINCIPALES DEL FIRMWARE 40
A.1 Adquisición y filtrado de la entrada 40
A.2 Modelo discreto y temporización 40
A.3 Implementación del retardo 41
A.4 Alteraciones de la medición 42
A.5 Apagado y modo de bajo consumo 42

ANEXO B – GUÍA PARA FUTURAS MODIFICACIONES 43

1. INTRODUCCIÓN

1.1. Problemática y necesidad identificada

El estudio de los lazos de control requiere observar cómo responde un proceso frente a las acciones generadas por un controlador. Sin embargo, en actividades de capacitación y pruebas de laboratorio es frecuente disponer de un PLC con funciones de control PID, pero no de una planta física sobre la cual trabajar.

Una alternativa sencilla consiste en forzar manualmente el valor de la variable de proceso. Este procedimiento puede servir para comprobar la lógica del programa o la visualización de las variables, pero no permite estudiar adecuadamente el comportamiento del lazo. La variable forzada no responde a la salida del controlador ni presenta una dinámica propia, por lo que el lazo no funciona de manera cerrada.

También existen calibradores capaces de generar y medir señales industriales, pero su función principal es verificar instrumentos y circuitos. Por otra parte, las herramientas de simulación ejecutadas en una computadora pueden representar procesos más complejos, aunque requieren equipamiento adicional y una configuración de comunicación compatible con el controlador.

A partir de esta necesidad se plantea el desarrollo de un equipo que reciba la señal generada por el PLC, calcule la respuesta de un proceso simulado y devuelva la variable resultante mediante una señal analógica. De esta manera, el controlador puede operar sobre una respuesta dinámica sin necesidad de disponer de una planta física.

El equipo se orienta principalmente a la capacitación y experimentación con lazos de control. Como aplicación secundaria, puede utilizarse en demostraciones técnicas y verificaciones funcionales básicas.

1.2. Objetivos

El objetivo principal del presente proyecto es diseñar e implementar un simulador de variables de proceso que permita cerrar un lazo de control entre un PLC real y un modelo dinámico ejecutado en un sistema embebido, utilizando señales analógicas de 4–20 mA.

Se establecen como objetivos del proyecto:

- Diseñar una etapa electrónica capaz de adquirir una señal de 4–20 mA proveniente de la salida analógica de un PLC.
- Diseñar una etapa capaz de generar una señal de 4–20 mA compatible con la entrada analógica del controlador.
- Implementar un modelo de proceso de primer orden con ganancia, constante de tiempo y retardo configurables.
- Incorporar modos de operación automático y manual.
- Permitir la aplicación de offset, ruido de medición y congelamiento sobre la variable simulada.

- Incorporar configuraciones preestablecidas que faciliten la selección inicial de diferentes comportamientos dinámicos.
- Implementar una interfaz local mediante pantalla y pulsadores, y una interfaz web mediante Wi-Fi.
- Registrar las principales variables y parámetros de funcionamiento en archivos CSV mediante una tarjeta microSD.
- Integrar las distintas etapas en un prototipo funcional y comprobar su operación conectado a un PLC.

1.3. Alcance y limitaciones

El proyecto comprende el diseño, construcción y programación de un prototipo funcional de un simulador de variables de proceso basado en un ESP32-S3-WROOM-1-N16R8 devkit.

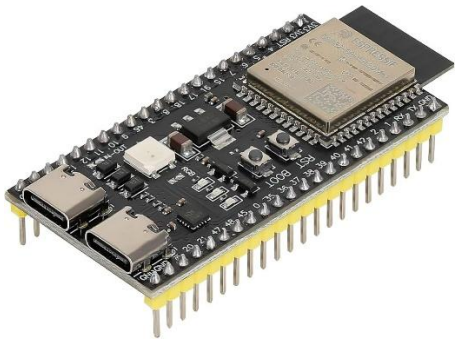


Figura 1.1 – Microcontrolador ESP32-S3 devkit

La versión desarrollada dispone de una entrada y una salida de 4–20 mA, correspondientes a un único lazo de control. La señal proveniente de la salida analógica del PLC representa la variable manipulada. El simulador adquiere esta señal, calcula la respuesta del modelo y genera la corriente que el PLC recibe como variable de proceso.

El proceso se representa mediante un modelo de primer orden con retardo. El usuario puede modificar directamente la ganancia K, la constante de tiempo τ y el retardo θ. También puede aplicar offset, ruido de medición y congelamiento sobre la salida calculada.

Los presets denominados Nivel, Caudal, Presión y Temperatura no representan modelos físicos diferentes. Cada uno carga una combinación inicial de parámetros sobre el mismo modelo matemático.

El prototipo incluye:

- Operación automática y manual.
- Pantalla OLED y pulsadores.
- Interfaz web mediante Wi-Fi.
- Almacenamiento de configuraciones.
- Registro de datos en una tarjeta microSD.

- Funciones de calibración para las señales analógicas.

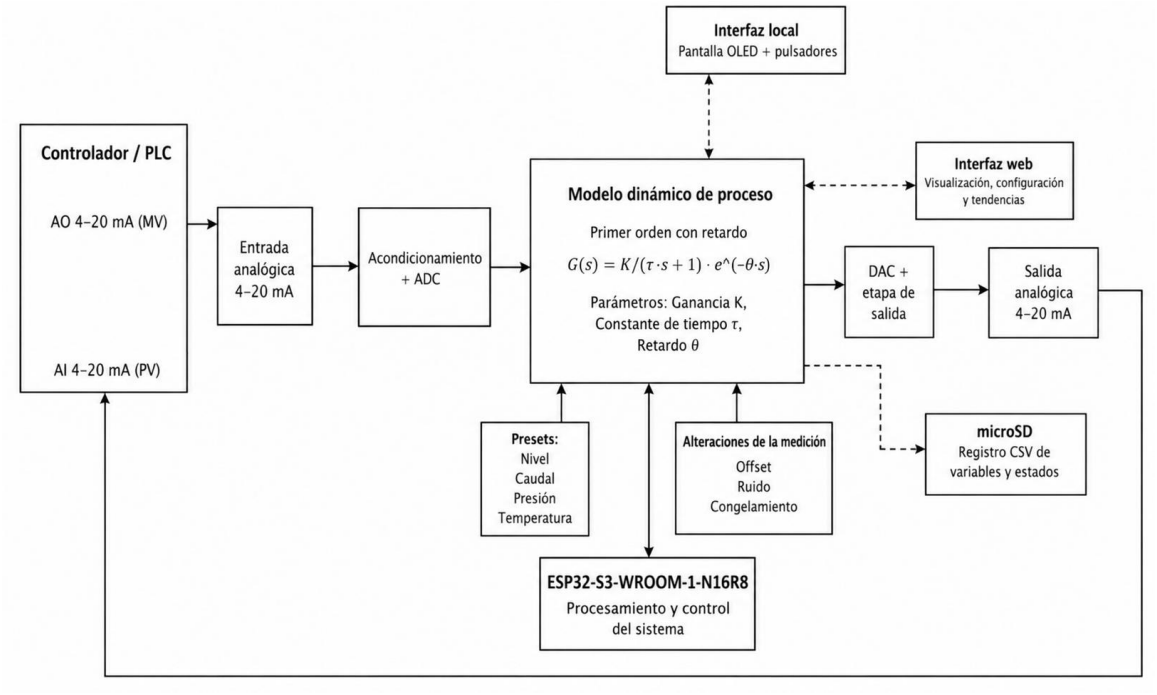


Figura 1.2 – Arquitectura general del simulador de variables de proceso

La versión actual no incorpora aislación galvánica entre las señales analógicas y el sistema de procesamiento. Tampoco permite simular varios lazos en forma simultánea ni representa fenómenos específicos de una planta, como no linealidades, interacciones entre variables o características particulares de sensores y actuadores.

El equipo no fue desarrollado para reemplazar un calibrador profesional ni una plataforma industrial de simulación. Su alcance corresponde a una herramienta de capacitación y experimentación con un único lazo de control.

2. FUNDAMENTOS TEÓRICOS

En este capítulo se presentan los conceptos necesarios para comprender el funcionamiento del simulador. Las decisiones específicas de diseño electrónico, la implementación del firmware y la configuración del PLC se desarrollan posteriormente en sus respectivos capítulos.

2.1. Normalización de las señales

El simulador recibe y genera señales de corriente de 4–20 mA. Para realizar los cálculos, estas señales se convierten internamente a una escala normalizada entre 0 % y 100 %.

La relación utilizada para convertir corriente a porcentaje es:

$$x[\%] = \left[\frac{I[mA] - 4mA}{16mA}\right] \cdot 100$$

La conversión inversa se expresa como:

$$I[mA] = 4mA + 16mA \cdot \frac{PV[\%]}{100}$$

Por lo tanto, 4 mA representan el 0 % de la escala y 20 mA representan el 100 %. Esta normalización permite ejecutar el modelo sin depender directamente de los valores eléctricos utilizados en la entrada y la salida.

Las conversiones realizadas por el ADC y el DAC, junto con sus correspondientes calibraciones, se desarrollan en los capítulos de hardware y firmware.

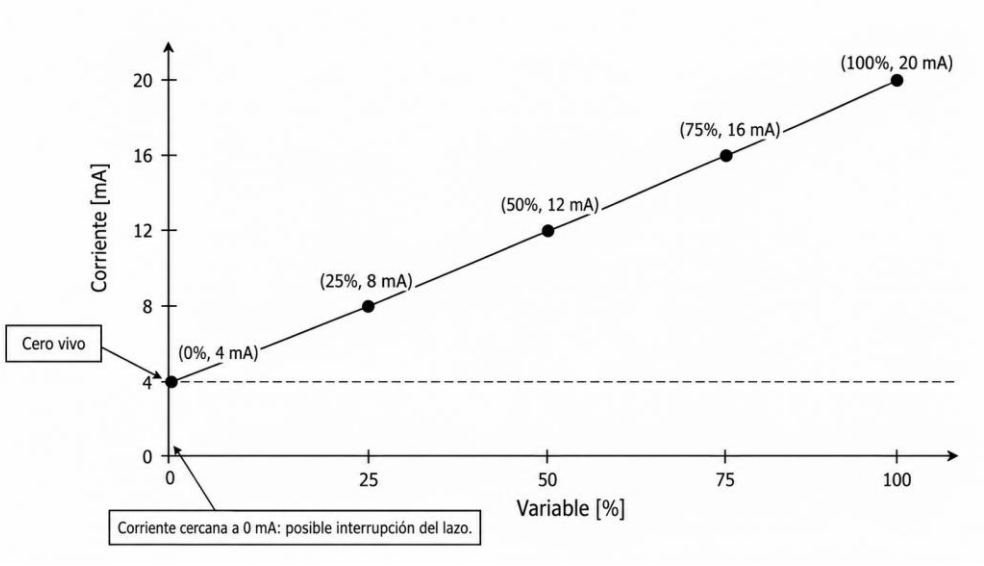


Figura 2.1 – Representación de una variable mediante una señal 4–20 mA

2.2. Modelo de primer orden con tiempo muerto

El proceso simulado se representa mediante un modelo de primer orden con retardo:

Al incorporar el tiempo muerto, la función de transferencia queda expresada como:

$$G(s) = \left[\frac{K}{\tau \cdot s + 1}\right] \cdot e^{-\theta \cdot s}$$

donde:

- K es la ganancia del proceso.
- τ es la constante de tiempo, expresada en segundos.
- θ es el retardo, expresado en segundos.

La ganancia establece la relación entre la variación de entrada y el valor final alcanzado por la salida:

$$K = \frac{\Delta y}{\Delta u}$$

Como la entrada y la salida se expresan en porcentaje, K es adimensional. Para K = 1, una variación de entrada produce una variación final de igual magnitud en la salida. Valores mayores o menores modifican esta relación.

La constante de tiempo determina la rapidez de la respuesta. Un valor pequeño de τ produce una evolución más rápida, mientras que un valor mayor genera una respuesta más lenta.

El retardo θ establece el tiempo que transcurre entre una modificación de la entrada y el comienzo de su efecto sobre la salida.

Para una entrada escalón y condiciones iniciales nulas, la respuesta puede expresarse como:

$$y(t) = 0, \qquad para\ t < \theta$$

$$y(t) = K \cdot \Delta u \cdot \left[1 - e^{-\frac{t-\theta}{\tau}}\right], para\ t \geq \theta$$

Una vez transcurrido el retardo, la salida evoluciona de forma exponencial. En el instante t = θ + τ alcanza aproximadamente el 63,2 % de su variación final.

2.3. Implementación discreta

El modelo se ejecuta en el ESP32-S3 y, por lo tanto, sus variables se actualizan en intervalos discretos separados por un período Ts.

La ecuación utilizada para el modelo de primer orden es:

$$y[k] = a \cdot y[k-1] + b \cdot u[k-N]$$

donde:

- y[k] es la salida calculada en la muestra actual.
- y[k−1] es la salida calculada en la muestra anterior.
- u[k−N] es la entrada retardada.
- N es la cantidad de muestras utilizadas para representar el retardo.

Los coeficientes del modelo se calculan como:

$$a = e^{-\frac{T_s}{\tau}}$$

$$b = K \cdot (1 - a)$$

El retardo se convierte en una cantidad entera de muestras:

$$N = redondear\left(\frac{\theta}{T_s}\right)$$

Para aplicar este retardo, las muestras de entrada se almacenan en un buffer circular. En cada actualización se incorpora el valor actual y se recupera la muestra correspondiente a N períodos anteriores.

El retardo efectivo queda limitado a múltiplos enteros del período de actualización. Su valor puede expresarse como $\theta_d = N \cdot T_s$. Por lo tanto, puede existir una diferencia entre el retardo configurado y el aplicado por el modelo.

3. ARQUITECTURA Y FUNCIONAMIENTO GENERAL

Este capítulo describe la organización del sistema y el recorrido de las señales. Los circuitos electrónicos y la implementación del firmware se desarrollan en los capítulos siguientes.

3.1. Conexión con el PLC

El simulador se conecta entre una salida analógica y una entrada analógica del PLC. La salida del controlador representa la variable manipulada y se utiliza como entrada del modelo. La respuesta calculada por el simulador vuelve al PLC como variable de proceso.

El recorrido de las señales es:

PLC AO 4–20 mA → entrada del simulador → modelo del proceso → salida del simulador 4–20 mA → PLC AI

La entrada analógica del PLC utiliza la señal recibida como realimentación del controlador PID. De esta manera, la acción de control produce una respuesta dinámica calculada por el simulador.

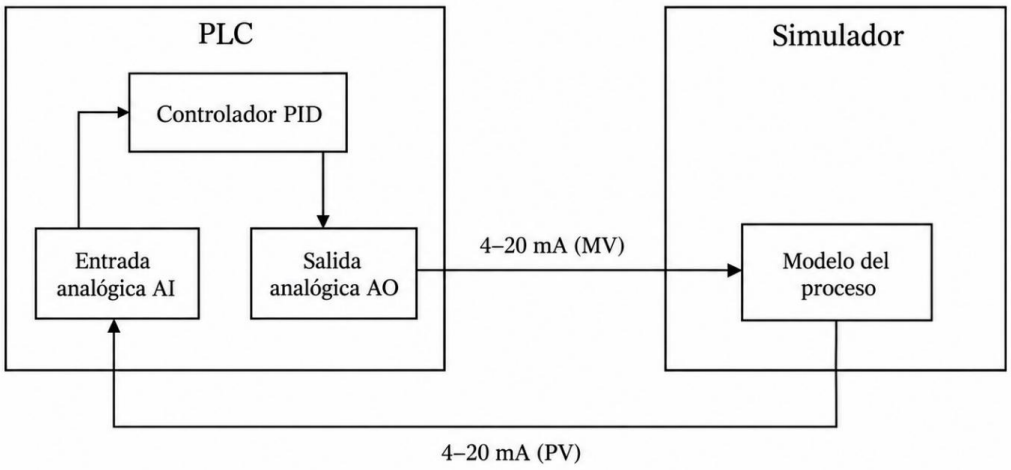


Figura 3.1 – Conexión general entre el PLC y el simulador

3.2. Organización interna y temporización

La señal de corriente proveniente del PLC ingresa al circuito de entrada, donde se convierte en una tensión adecuada para el ADC del ESP32-S3. El firmware calibra y normaliza esta lectura antes de utilizarla como entrada del modelo.

El ESP32-S3 calcula la respuesta del proceso y genera una referencia digital para el DAC MCP4725. La tensión obtenida controla el circuito de salida encargado de producir la señal de 4–20 mA enviada al PLC.

El ESP32-S3 también administra la pantalla OLED, los pulsadores, la conexión Wi-Fi, el buzzer y la tarjeta microSD.

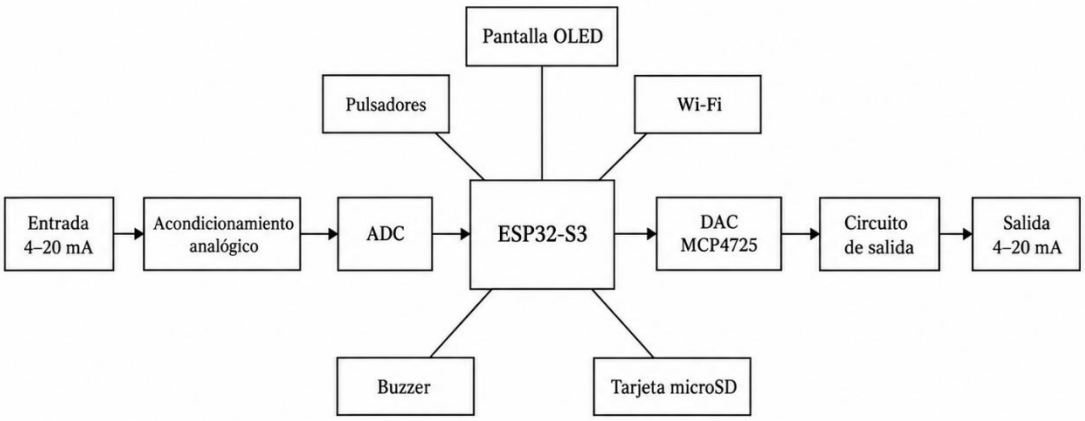


Figura 3.2 – Arquitectura interna del simulador

Temporización

La adquisición de la entrada y la actualización del modelo se ejecutan con períodos diferentes. Los valores nominales utilizados se resumen en la Tabla 3.1.

Proceso	Período nominal	Frecuencia nominal
Adquisición de la entrada	2 ms	500 Hz
Modelo y salida analógica	50 ms	20 Hz
Registro en microSD	1 s	1 Hz

Tabla 3.1 – Períodos de actualización del sistema

En cada actualización de la entrada se realizan cuatro conversiones consecutivas del ADC. Las lecturas se promedian y el resultado se procesa mediante un filtro digital IIR.

Cada 50 ms, el modelo utiliza el último valor filtrado disponible, calcula el nuevo estado y actualiza la salida mediante el DAC. Por lo tanto, durante un período del modelo pueden realizarse aproximadamente 25 actualizaciones de la entrada. El modelo no procesa estas muestras por separado, sino que utiliza el valor más reciente.

La ejecución del modelo utiliza un período discreto nominal de 50 ms. La referencia temporal se mantiene de forma acumulativa, de manera que eventuales demoras producidas por otras tareas del firmware no modifican la escala temporal utilizada en el cálculo del proceso. De esta forma, los parámetros de constante de tiempo y retardo conservan su correspondencia con los valores configurados.

3.3. Modos de funcionamiento y configuración

Habilitación de la salida

La habilitación de la salida es independiente del modo de funcionamiento seleccionado. Mientras se encuentra deshabilitada, el firmware escribe cero cuentas en el DAC y no genera la señal normal de 4–20 mA.

Esta condición debe diferenciarse de una salida configurada en 0 %. Cuando la salida está habilitada, el 0 % de escala corresponde a 4 mA.

Modo automático

En modo automático, la señal recibida desde el PLC se utiliza como entrada del modelo. La respuesta calculada se procesa y se convierte en la corriente enviada a la entrada analógica del controlador.

El recorrido en este modo es:

Entrada medida → modelo del proceso → alteraciones de medición → salida 4–20 mA

Modo manual

En modo manual, el usuario establece directamente el porcentaje de salida. La entrada proveniente del PLC y la dinámica del modelo no intervienen en el cálculo.

Al pasar de modo automático a manual, el último valor de salida se utiliza como valor manual inicial para reducir el cambio producido durante la transición.

Las denominaciones automático y manual corresponden al funcionamiento del simulador y no al modo de operación del controlador PID del PLC.

Parámetros configurables

El usuario puede modificar directamente los parámetros del modelo y las alteraciones aplicadas sobre la medición. Los rangos disponibles se indican en la Tabla 3.2.

Parámetro	Rango	Unidad
Ganancia K	0,0 a 5,0	Adimensional
Constante de tiempo τ	0,1 a 60,0	s
Retardo θ	0 a 30	s
Offset	−25 a +25	%
Ruido de medición	0 a 10	%
Congelamiento	Activado o desactivado	—

Tabla 3.2 – Parámetros configurables

Alteraciones de la medición

El offset se suma directamente al resultado del modelo y se expresa como porcentaje de la escala. Por ejemplo, un offset de +5 % agrega cinco puntos porcentuales a la medición simulada.

El valor configurado como ruido representa una variación aleatoria alrededor del valor de salida, expresada en puntos porcentuales de la escala. Por ejemplo, para un valor de salida de 50 % y un ruido configurado de ±10 puntos porcentuales, la salida puede variar aleatoriamente entre 40 % y 60 %.

Al activar el congelamiento, el firmware captura el primer valor calculado después de la activación y lo mantiene hasta que la función se deshabilita.

El orden de procesamiento es:

Salida base → limitación del proceso entre 0 % y 100 % → offset → ruido → congelamiento → limitación final entre 0 % y 100 %.

En modo automático, la salida base corresponde al resultado del modelo. En modo manual, corresponde al valor establecido por el usuario.

Configuraciones preestablecidas

El simulador incorpora cuatro configuraciones preestablecidas. Cada una carga valores iniciales de ganancia, constante de tiempo y retardo sobre el mismo modelo matemático.

Preset	Ganancia K	Tau τ [s]	Retardo θ [s]
Nivel	1,00	10,0	0,8
Caudal	1,00	2,0	0,2
Presión	1,00	5,0	0,5
Temperatura	1,00	20,0	2,0

Tabla 3.3 – Parámetros de las configuraciones preestablecidas

Estos valores funcionan como puntos de partida y pueden modificarse desde las interfaces. Cuando el usuario cambia K, τ o θ, la configuración pasa a identificarse como Personalizada.

Interfaces y registro

La tarjeta microSD permite registrar las principales variables y parámetros de funcionamiento en archivos CSV con un período nominal de un segundo. El funcionamiento interno del registro y la gestión de los archivos se describen en la sección 5.5.

3.4. Determinación de los presets

Los presets incluidos en el simulador fueron definidos con el objetivo de representar comportamientos dinámicos diferenciados para uso didáctico.

Durante el desarrollo se analizaron registros históricos de distintos lazos industriales como referencia para comparar órdenes de magnitud y velocidades de respuesta. Sin embargo, debido a que dichos registros no correspondían a ensayos específicos de identificación de procesos, no se utilizaron directamente para obtener los parámetros finales.

Se adoptó una ganancia K = 1 para todos los presets, de manera que el modelo pueda utilizar el rango completo de 0 % a 100 % sin limitar artificialmente la salida. Las diferencias entre cada configuración se establecen principalmente mediante la constante de tiempo τ y el retardo θ.

De esta manera, Caudal presenta la respuesta más rápida, seguido por Presión y Nivel, mientras que Temperatura representa la dinámica más lenta. Estos valores deben interpretarse como configuraciones representativas para prácticas y experimentación, y no como modelos identificados de procesos industriales específicos.

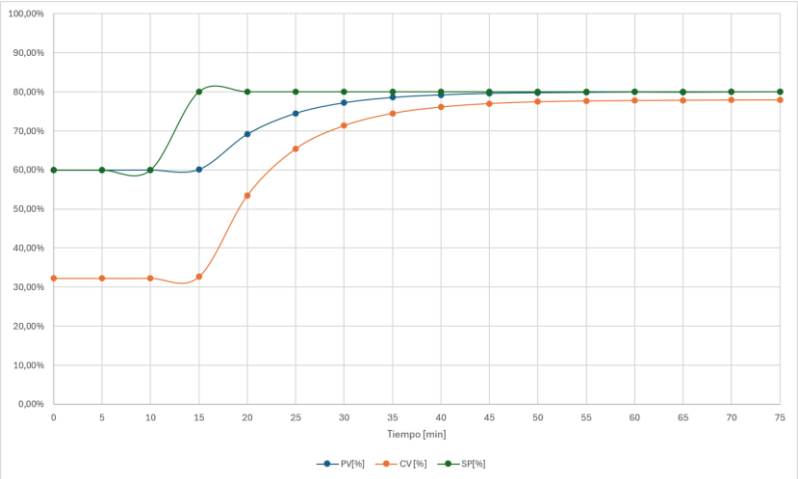


Figura 3.3 – Ejemplo de respuesta histórica de un lazo de control industrial

4. DESARROLLO DEL HARDWARE

El hardware fue diseñado en Proteus v8.17 SP2, y su distribución fue en dos placas, una placa base que contiene los circuitos de procesamiento, alimentación y señales analógicas, y una placa superior destinada a la operación local del equipo.

4.1. Unidad de procesamiento y alimentación

Unidad de procesamiento

El sistema utiliza una placa de desarrollo basada en el módulo ESP32-S3-WROOM-1-N16R8 devkit. Esta variante dispone de 16 MB de memoria flash, 8 MB de PSRAM y conectividad Wi-Fi.

Las conexiones utilizadas se indican en la Tabla 4.1.

Función	Interfaz	GPIO
Entrada analógica desde el PLC	ADC1_CH5	6
Datos I²C	SDA	8
Reloj I²C	SCL	9
Selección de microSD	CS	10
Datos hacia microSD	MOSI	11
Reloj de microSD	SCK	12
Datos desde microSD	MISO	13
Pulsador ON/OFF	Entrada digital	15
Buzzer	Salida PWM	21
Pulsador Menú	Entrada digital	5
Pulsador Subir	Entrada digital	7
Pulsador Bajar	Entrada digital	16
Pulsador Enter	Entrada digital	17

Tabla 4.1 – Asignación de entradas, salidas e interfaces

El GPIO 6 se utiliza como entrada del ADC1, mientras que la pantalla OLED y el DAC MCP4725 comparten el bus I²C. La tarjeta microSD utiliza una interfaz SPI independiente.

Alimentación

El equipo recibe una alimentación externa de 24 V. Esta tensión se utiliza en el circuito de salida de corriente y se reduce mediante dos etapas para obtener las alimentaciones de 5 V y 3,3 V. La línea de 5 V alimenta la placa de desarrollo y el buzzer. La línea de 3,3 V alimenta los circuitos lógicos, el amplificador de entrada, la pantalla OLED y el DAC.

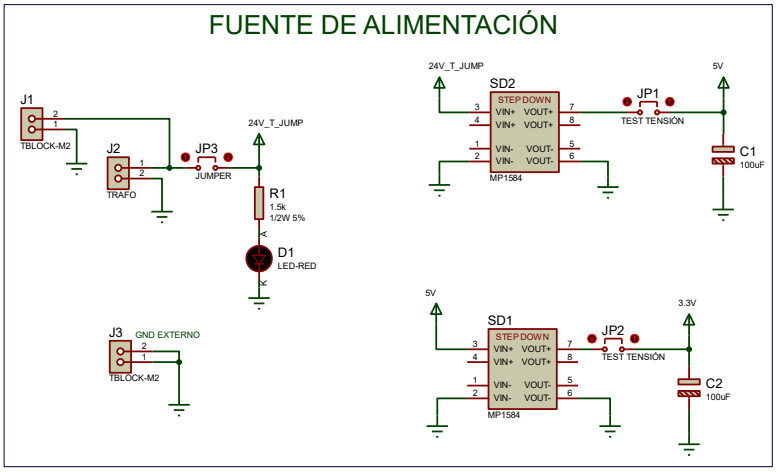


Figura 4.1 – Circuito de alimentación y unidad de procesamiento

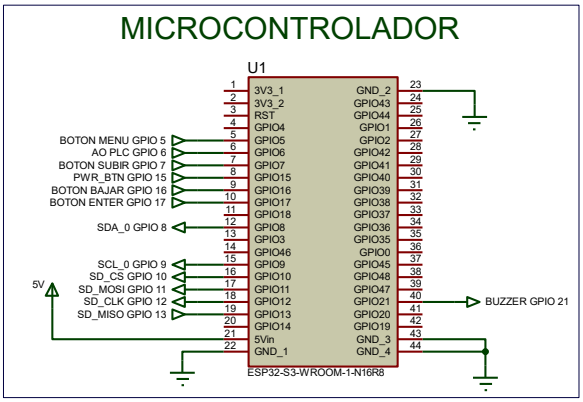


Figura 4.2 – Conexiones principales del ESP32-S3

Los capacitores C1 y C2, de 100 µF, proporcionan filtrado local sobre las líneas de alimentación. El LED D1 indica la presencia de 24 V.

Para generar las alimentaciones de 5 V y 3,3 V se utilizaron dos módulos reductores DC-DC iguales, basados en el integrado MP1584. El primer módulo reduce la tensión de 24 V a 5 V, mientras que el segundo convierte los 5 V en 3,3 V.

El MP1584 admite una tensión de entrada de 4,5 V a 28 V y una corriente de salida de hasta 3 A. La frecuencia de conmutación puede ajustarse entre 100 kHz y 1,5 MHz. Para el módulo comercial utilizado se indica una eficiencia máxima del 92 %.

Tanto la corriente de 3 A como la eficiencia del 92 % corresponden a valores máximos especificados. La eficiencia real depende de las tensiones de entrada y salida, la corriente consumida y las condiciones térmicas, y no fue medida específicamente en el equipo desarrollado.

Consumo del equipo

El consumo total se midió sobre la alimentación de 24 V, con el multímetro conectado en serie con la entrada del equipo. Se evaluaron diferentes condiciones de funcionamiento para observar la influencia de la conexión Wi-Fi, la escritura en la tarjeta microSD y la salida analógica.

Condición	Corriente medida [mA]
Equipo apagado (bajo consumo)	20,92
Wi-Fi deshabilitado, sin captura y salida deshabilitada	40,5
Wi-Fi activo, sin captura y salida deshabilitada	53,5
Wi-Fi activo, captura microSD activa y salida deshabilitada	56,4
Wi-Fi activo, captura microSD activa y salida al 100 %	74,1

Tabla 4.2 – Consumo del equipo desde la alimentación de 24 V

La mayor corriente medida fue de 74,1 mA, correspondiente al funcionamiento con Wi-Fi activo, registro en microSD y salida analógica configurada al 100 %. La potencia absorbida desde la fuente de 24 V en esta condición es:

$$P = V \cdot I$$

$$P = 24V \cdot 0{,}0741A = 1{,}78W$$

Las mediciones muestran que la activación del Wi-Fi incrementa el consumo respecto del estado básico, mientras que la escritura en la tarjeta microSD produce un aumento menor. La mayor variación se observa al habilitar la salida analógica a 20 mA.

4.2. Circuito de entrada de 4–20 mA

El circuito de entrada convierte la corriente proveniente del PLC en una tensión adecuada para el ADC del ESP32-S3.

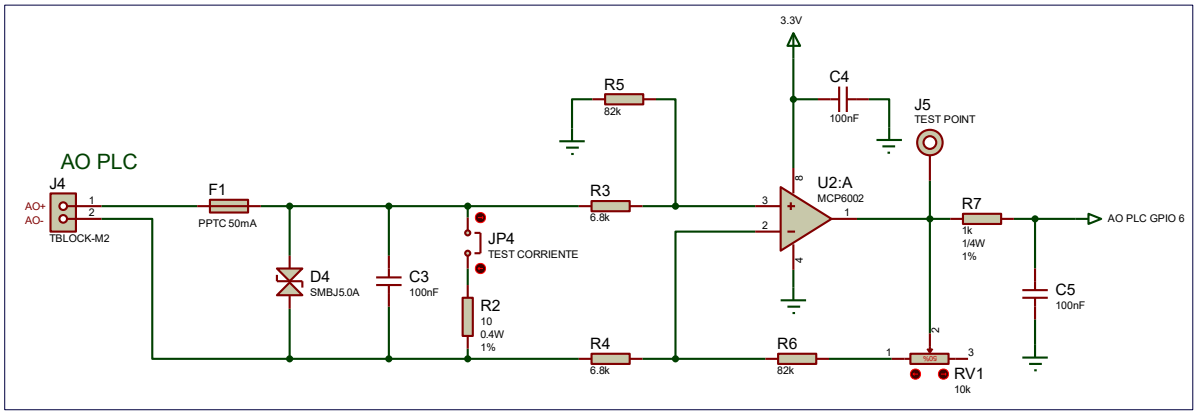


Figura 4.3 – Circuito de entrada de 4–20 mA

Conversión de corriente a tensión

La corriente de entrada circula por la resistencia shunt R2, de 10 Ω. La tensión generada es:

$$V_{SHUNT} = I \cdot R_2$$

Para 4 mA:

$$V_{SHUNT} = 0{,}004A \cdot 10\Omega = 0{,}040V$$

Para 20 mA:

$$V_{SHUNT} = 0{,}020A \cdot 10\Omega = 0{,}200V$$

La entrada queda inicialmente convertida en un rango de 40 a 200 mV. La caída máxima introducida en el lazo es de 200 mV.

La potencia máxima sobre R2 ocurre a 20 mA:

$$P_{R_2} = I^2 \cdot R_2$$

$$P_{R_2} = (0{,}020A)^2 \cdot 10\Omega = 0{,}004W$$

La resistencia instalada es de 0,4 W, por lo que trabaja con un margen amplio.

Acondicionamiento de la señal

La tensión de R2 ingresa al terminal no inversor del MCP6002 mediante R3, de 6,8 kΩ. R5, de 82 kΩ, completa el divisor:

$$V+= V_{SHUNT} \cdot \frac{R_5}{R_3 + R_5} = V_{SHUNT} \cdot \frac{82k\Omega}{6{,}8k\Omega + 82k\Omega}$$

$$V+= 0{,}9234 \cdot V_{SHUNT}$$

El operacional se configura como amplificador no inversor. Sin considerar el ajuste de RV1, su ganancia es:

$$A_V = 1 + \frac{R_6}{R_4} = 1 + \frac{82k\Omega}{6{,}8k\Omega} = 13{,}059$$

La ganancia total entre R2 y la salida del operacional resulta:

$$A_{TOTAL} = 0{,}9234 \cdot 13{,}059 = 12{,}059$$

Los valores teóricos se indican en la Tabla 4.3.

Corriente de entrada	Tensión en R2	Salida del MCP6002
4 mA	0,040 V	0,482 V
12 mA	0,120 V	1,447 V
20 mA	0,200 V	2,412 V

Tabla 4.3 – Valores teóricos del circuito de entrada

RV1, de 10 kΩ, permite realizar un ajuste fino de la ganancia. El valor definitivo se establece experimentalmente durante la calibración del circuito. La calibración final se realiza aplicando corrientes conocidas y almacenando las cuentas del ADC correspondientes a 4 mA y 20 mA.

Filtrado y protección

R7, de 1 kΩ, y C5, de 100 nF, forman un filtro pasa bajos antes del ADC:

$$f_C = \frac{1}{2 \cdot \pi \cdot R_7 \cdot C_5} = \frac{1}{2 \cdot \pi \cdot 1k\Omega \cdot 100nF} = 1{,}59kHz$$

El filtro atenúa componentes de alta frecuencia antes de la conversión. R7 también limita la corriente hacia el GPIO frente a una condición anómala.

C3, de 100 nF, se encuentra en paralelo con la entrada. Considerando R2 como resistencia equivalente aproximada:

$$f_C = \frac{1}{2 \cdot \pi \cdot 10\Omega \cdot 100nF} = 159kHz$$

Por su frecuencia de corte, este capacitor actúa principalmente sobre transitorios rápidos.

El circuito incorpora además:

- Un fusible rearmable de 50 mA.
- Un TVS SMBJ5.0A en paralelo con la entrada.
- Un capacitor de desacople de 100 nF para el MCP6002.
- Un punto de prueba sobre la salida amplificada.

La señal acondicionada ingresa al GPIO 6, correspondiente al canal ADC1_CH5. El firmware configura el ADC con una resolución de 12 bits y realiza una calibración de dos puntos.

4.3. Circuito de salida de 4–20 mA

El circuito de salida convierte la referencia generada por el DAC MCP4725 en una corriente regulada.

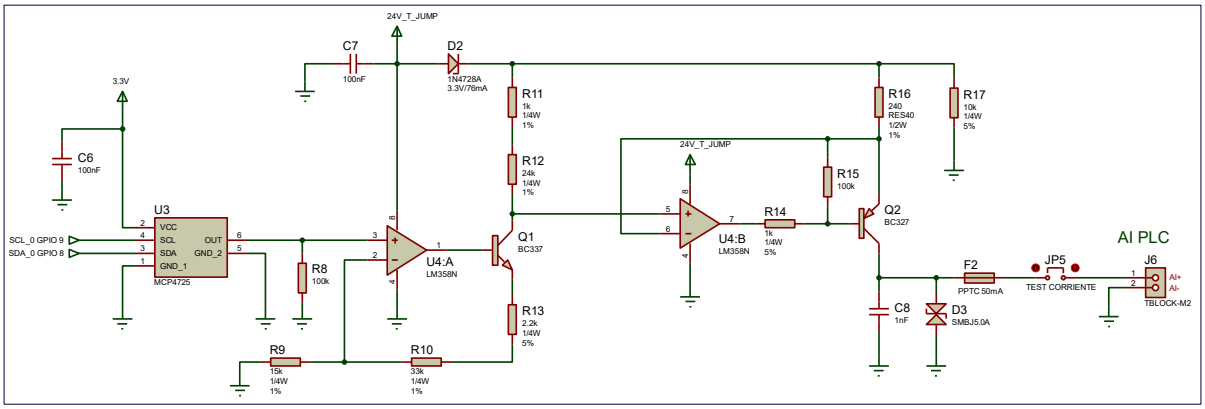


Figura 4.4 – Circuito de salida de 4–20 mA

Conversor digital-analógico

El MCP4725 posee una resolución de 12 bits y se alimenta con 3,3 V. La tensión ideal correspondiente a una cuenta N es:

$$V_{DAC} = 3{,}3\ V \cdot \frac{N}{4095}$$

Un capacitor de 100 nF proporciona desacople local y R8, de 100 kΩ, mantiene la salida referenciada a masa.

Primera etapa de control

U4:A controla la base de Q1 para mantener la tensión de su entrada inversora próxima a V_{DAC}. La corriente sobre R9, de 15 kΩ, es:

$$I_{R_9} = \frac{V_{DAC}}{R_9}$$

Despreciando la corriente de entrada del operacional y considerando pequeña la corriente de base de Q1:

$$I_{C1} \approx I_{E1} \approx \frac{V_{DAC}}{R_9}$$

R10 y R13 establecen la tensión de operación del emisor de Q1, mientras que R11 y R12 forman la rama conectada a su colector:

$$R_{11} + R_{12} = 1k\Omega + 24k\Omega = 25k\Omega$$

La caída generada en esta rama es:

$$V_{R_{11}} + R_{12} = I_{C_1} \cdot 25k\Omega$$

Etapa de salida

U4:B compara la tensión del colector de Q1 con la tensión del emisor de Q2. Mediante esta realimentación, el operacional controla Q2 para obtener aproximadamente la misma caída sobre R16 que sobre R11 y R12.

La corriente de salida puede expresarse como:

$$I_{SAL} \approx V_{DAC} \cdot \frac{R_{11} + R_{12}}{R_9 \cdot R_{16}}$$

Con R9 = 15 kΩ, R11 + R12 = 25 kΩ y R16 = 240 Ω:

$$I_{SAL} \approx V_{DAC} \cdot 6{,}944 mA/V$$

Los valores teóricos resultantes se muestran en la Tabla 4.4.

Corriente de salida	Tensión del DAC	Cuenta DAC teórica
4 mA	0,576 V	715
12 mA	1,728 V	2144
20 mA	2,880 V	3574

Tabla 4.4 – Valores teóricos del circuito de salida

Las cuentas se calculan mediante:

$$N = V_{DAC} \cdot \frac{4095}{3{,}3V}$$

Estos valores se utilizan como referencia inicial. La calibración de dos puntos permite compensar las tolerancias de los componentes y las diferencias respecto del modelo ideal.

Alimentación de la etapa de salida

El LM358 se alimenta desde la línea de 24 V. D2, un zener 1N4728A de 3,3 V, genera una referencia superior aproximada de:

$$V_P \approx 24\,V - 3{,}3V = 20{,}7V$$

R17, de 10 kΩ, mantiene una carga mínima sobre esta línea:

$$I_{R17} = \frac{20{,}7V}{10k\Omega} = 2{,}07mA$$

El valor real de V_P depende de la corriente y de la característica del zener, por lo que 20,7V debe considerarse una aproximación de diseño.

Disipación de potencia

La máxima potencia sobre R16 ocurre a 20 mA:

$$P_{R_{16}} = I^2 \cdot R_{16}$$

$$P_{R_{16}} = (0{,}020A)^2 \cdot 240\Omega = 0{,}096W$$

Se utilizó una resistencia de 0,5 W.

Para la entrada analógica utilizada en el PLC se considera una resistencia aproximada de 113 Ω. A 20 mA, la tensión sobre la carga es:

$$V_{PLC} = 0{,}020A \cdot 113\Omega = 2{,}26V$$

La tensión aproximada en el emisor de Q2 es:

$$V_{E2} = V_P - I_{SAL} \cdot R_{16}$$

$$V_{E2} = 20{,}7V - 0{,}020A \cdot 240\Omega = 15{,}9V$$

La potencia aproximada disipada por Q2 es:

$$P_{Q2} \approx (V_{E2} - V_{PLC}) \cdot I_{SAL}$$

$$P_{Q2} \approx (15{,}9V - 2{,}26V) \cdot 0{,}020A = 0{,}273W$$

Q2 presenta una disipación estimada mayor que R16, por lo que constituye el componente de mayor exigencia térmica dentro de la etapa de salida y debe considerarse especialmente durante el funcionamiento prolongado.

Protecciones

La salida incorpora:

- R14, que limita la corriente de base de Q2.
- R15, que mantiene Q2 apagado si el control de base queda indefinido.
- Un fusible rearmable de 50 mA.
- Un TVS SMBJ5.0A conectado entre la salida y masa.
- Un capacitor de 1 nF para transitorios de alta frecuencia.

- Un punto de medición de corriente.

Para mantener la tensión normal sobre la salida por debajo de los 5 V de trabajo del TVS, la carga conectada a 20 mA debe cumplir:

$$R_{CARGA} \leq \frac{5V}{0{,}020A} = 250\Omega$$

La entrada de 113 Ω utilizada con el PLC se encuentra dentro de este valor.

4.4. Interfaz local y periféricos

La interfaz local del simulador se encuentra distribuida entre la placa base y una placa superior destinada a la operación del equipo. Ambas placas se conectan mediante un cable IDC de diez conductores, a través del cual se transmiten la alimentación y las señales necesarias para la pantalla y los pulsadores.

La placa superior contiene los elementos directamente accesibles por el usuario, la pantalla OLED y los cinco pulsadores de operación. Parte de los circuitos asociados a estos elementos se encuentra en la placa base, donde se realiza el acondicionamiento de las entradas y la conexión con el ESP32-S3.

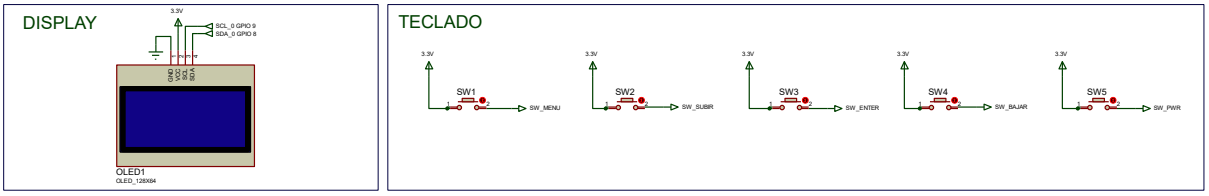


Figura 4.5 – Circuitos de la interfaz local en la placa superior

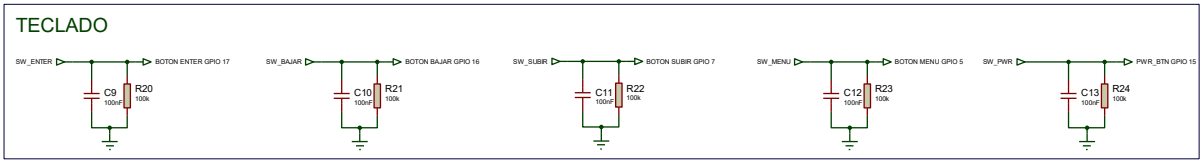


Figura 4.6 – Circuitos asociados a la interfaz local en la placa base

La conexión entre ambas placas se realiza mediante conectores IDC10, evitando el cableado individual de las señales y permitiendo desmontar la placa superior de forma independiente.

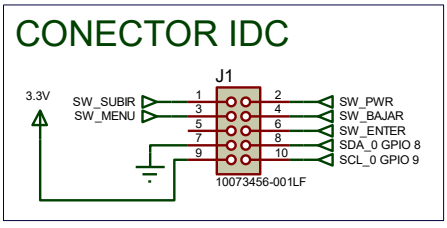


Figura 4.7 – Interconexión entre la placa base y la placa de interfaz

Pantalla OLED

La visualización local se realiza mediante una pantalla OLED monocromática de 1,3 pulgadas, con resolución de 128×64 píxeles y controlador SH1106. El módulo se comunica con el ESP32-S3 mediante el bus I2C y se alimenta desde la línea de 3,3 V.

Pulsadores

La interfaz dispone de los pulsadores Menú, Subir, Bajar, Enter y ON/OFF. Las entradas poseen resistencias de 100 kΩ hacia masa y capacitores de 100 nF para reducir perturbaciones y rebotes.

La constante de tiempo de cada entrada es:

$$\tau_{BOTÓN} = R \cdot C = 100k\Omega \cdot 100nF = 10ms$$

El filtrado físico se complementa mediante antirrebote implementado en el firmware.

El pulsador ON/OFF no interrumpe físicamente la alimentación general del equipo. El firmware deshabilita la salida analógica, finaliza la captura de datos y la comunicación Wi-Fi, detiene la tarjeta microSD, el buzzer y la pantalla OLED, y posteriormente coloca al ESP32-S3 en modo Deep-sleep.

Buzzer

El buzzer se encuentra montado en la placa base y se alimenta desde 5 V. Su accionamiento se realiza mediante un transistor NPN controlado por el ESP32-S3.

R18, de 1 kΩ, limita la corriente de base:

$$I_B \approx \frac{3{,}3V - 0{,}7V}{1k\Omega} = 2{,}6mA$$

Tarjeta microSD

El módulo microSD se encuentra en la placa base y se comunica con el ESP32-S3 mediante una interfaz SPI independiente. En la versión desarrollada, el bus se configura a 1 MHz y utiliza las conexiones indicadas en la Tabla 4.1.

4.5. Diseño de las placas e integración física

Los circuitos desarrollados se implementaron en dos placas de circuito impreso, una placa base y una placa superior correspondiente a la interfaz de usuario. Su distribución funcional y la interconexión entre ambas fueron descriptas en la sección anterior.

En esta sección se presenta la implementación física de ambas placas y su disposición final dentro del gabinete.

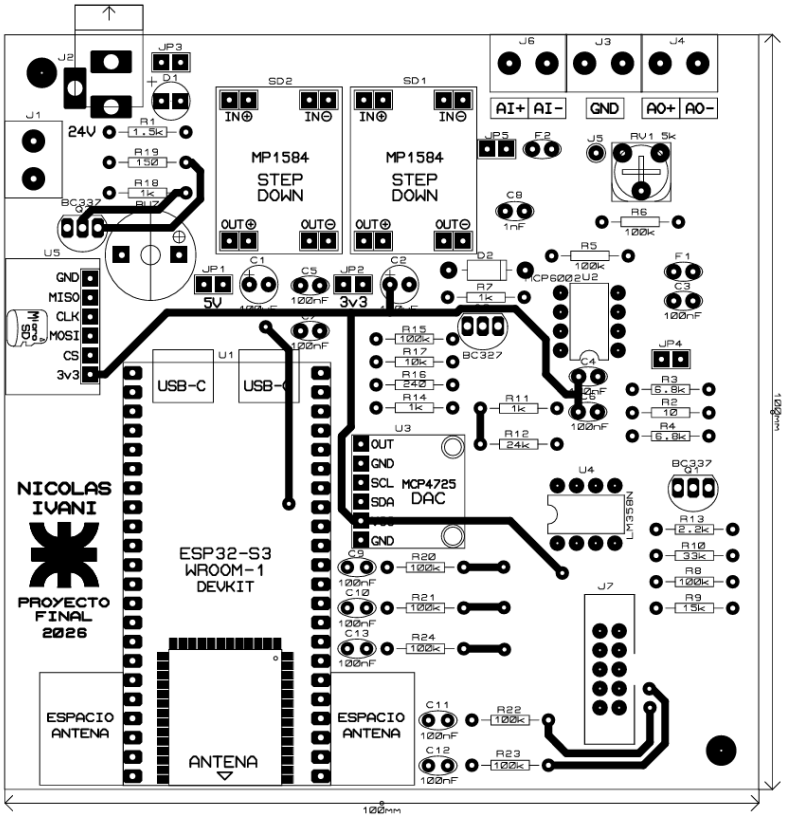


Figura 4.8 – Diseño PCB vista superior de la placa base

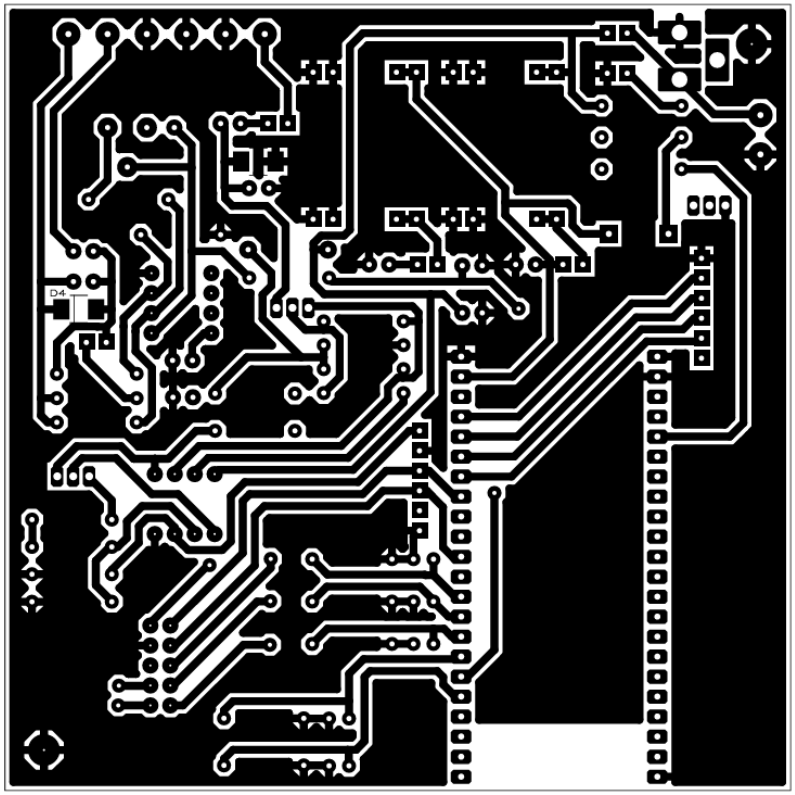


Figura 4.9 – Diseño PCB vista inferior de la placa base

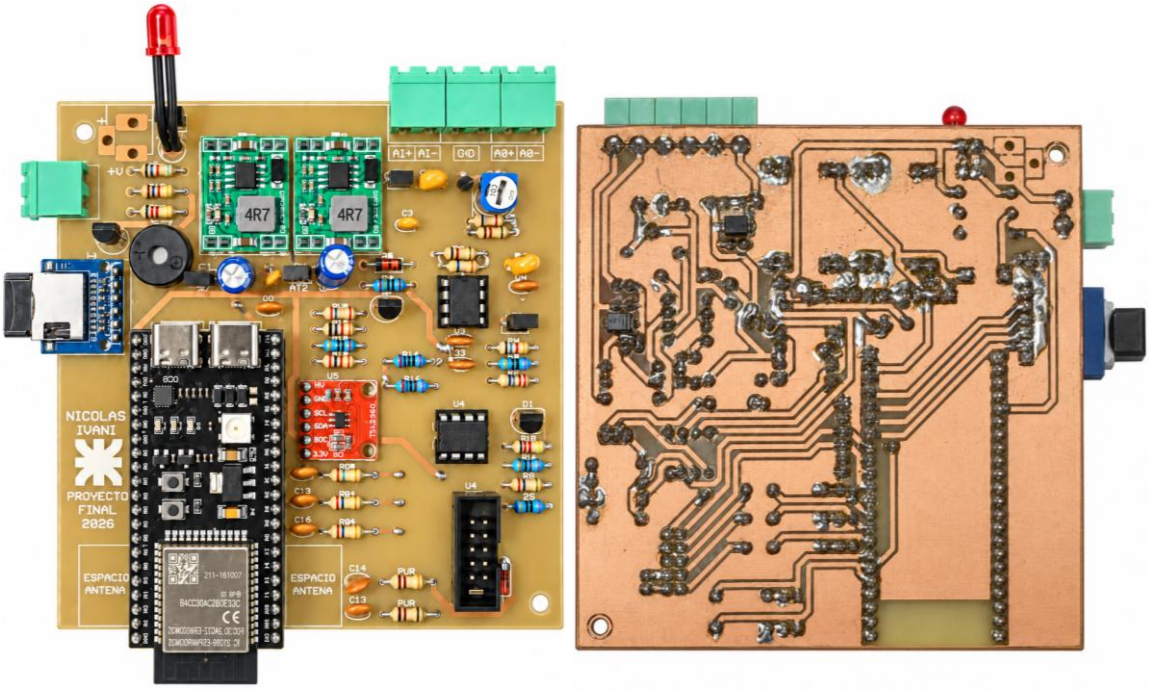


Figura 4.10 – Placa base fabricada vista superior e vista inferior

La placa superior contiene la pantalla OLED y los pulsadores de operación, permitiendo disponer los elementos de la interfaz sobre el frente del gabinete.

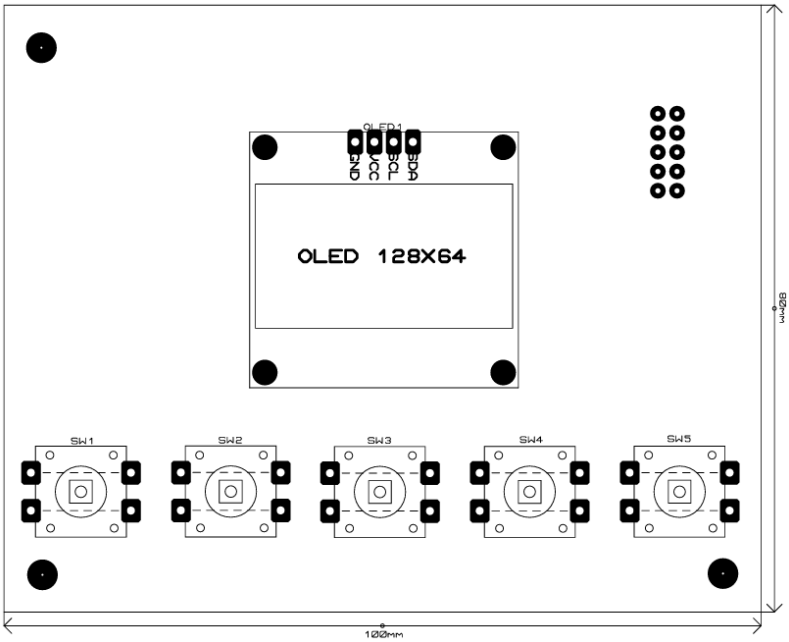


Figura 4.11 – Diseño PCB vista superior de la placa interfaz de usuario

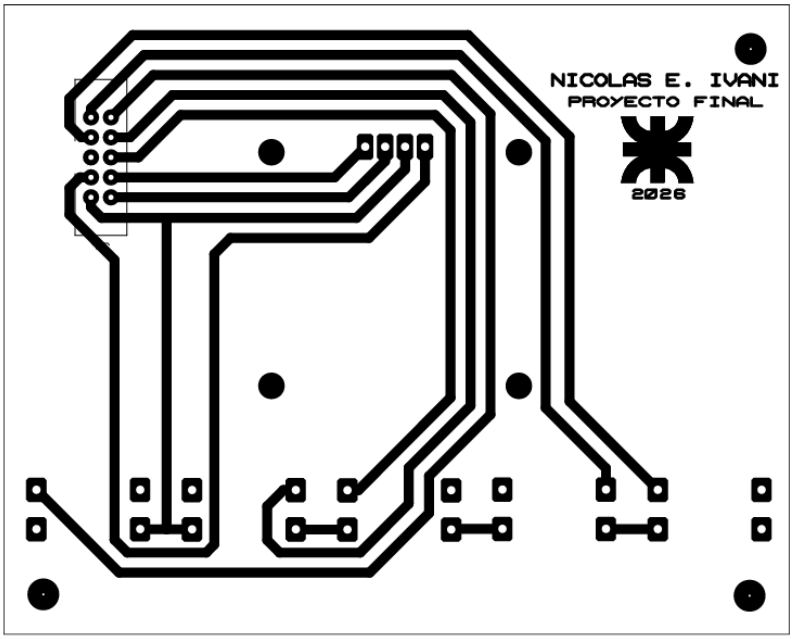


Figura 4.12 – Diseño PCB vista inferior de la placa interfaz de usuario

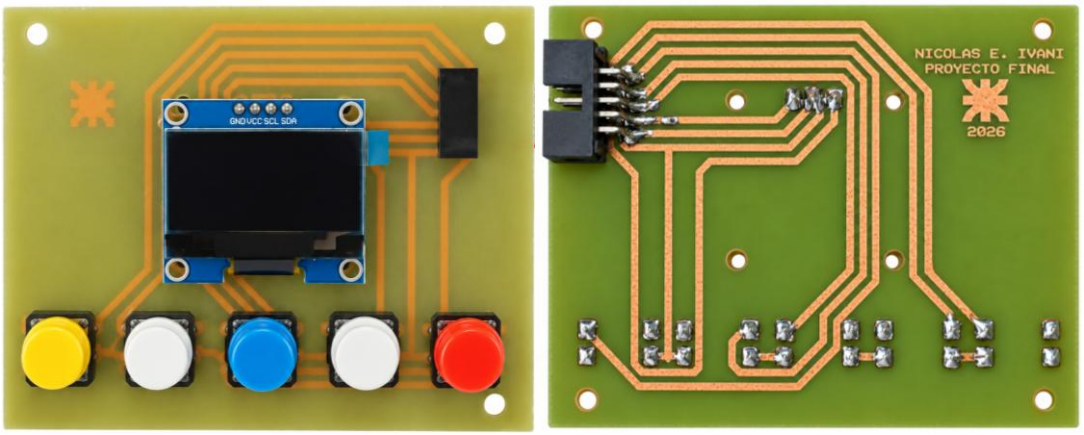


Figura 4.13 – Placa interfaz de usuario fabricada vista superior e vista inferior

El conjunto se montó dentro de un gabinete, dejando accesibles la pantalla, los pulsadores y los conectores necesarios para las señales analógicas y la alimentación.

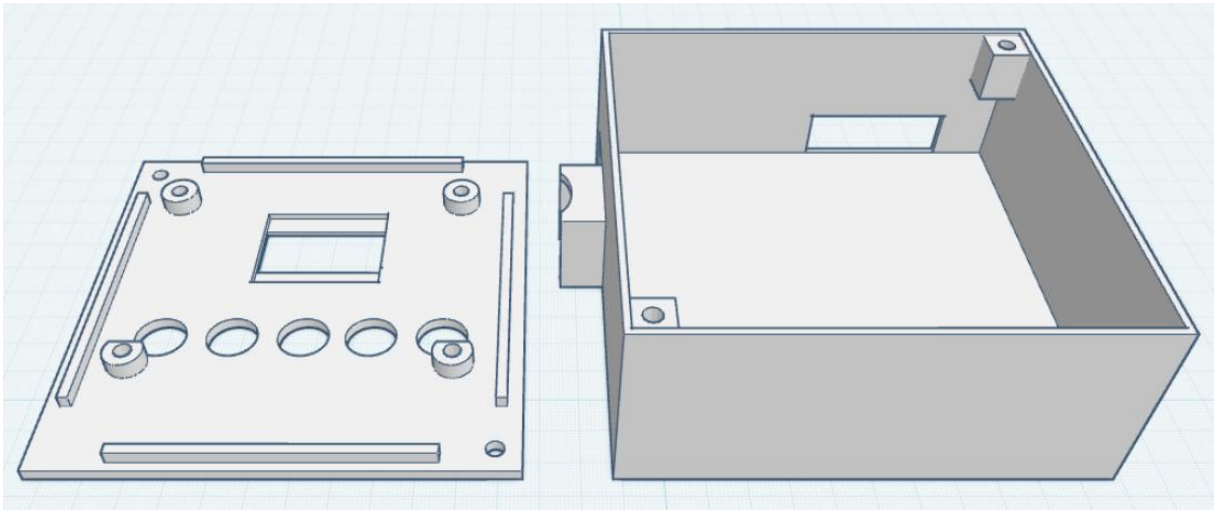


Figura 4.14 – Modelo 3D del gabinete del simulador

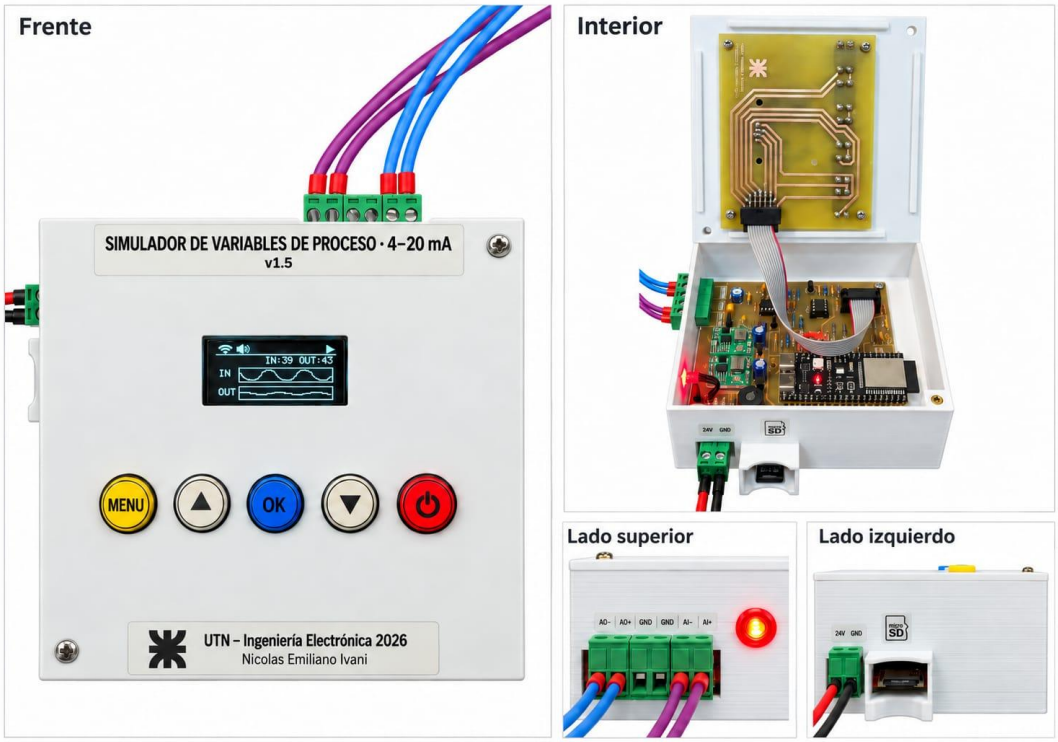


Figura 4.15 – Montaje final del hardware del simulador

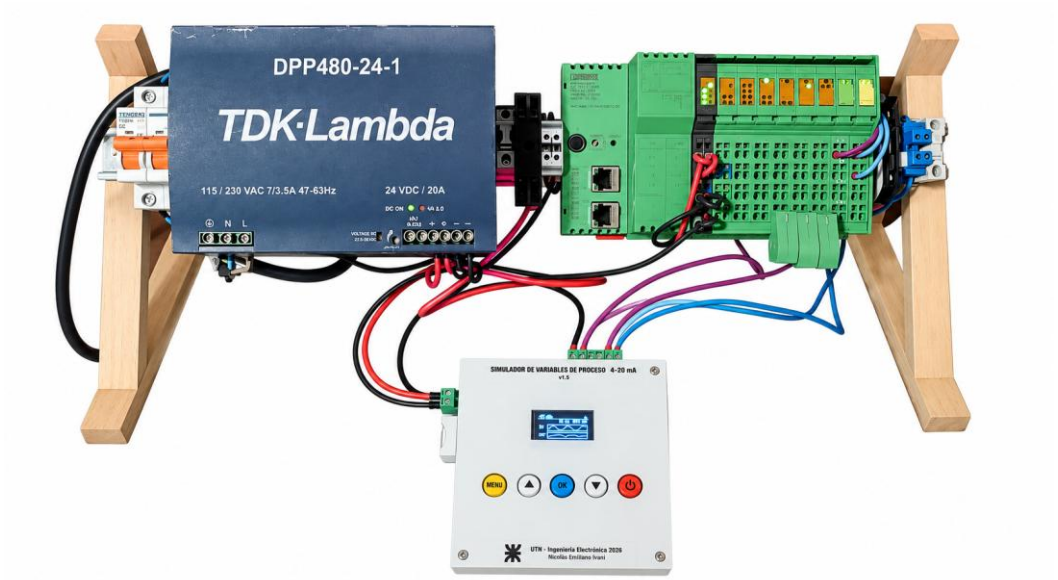


Figura 4.16 – Montaje final del banco de pruebas del PLC con el simulador

El módulo ESP32-S3 utiliza una antena integrada. Durante el diseño se mantuvo libre la zona correspondiente a la antena, evitando la presencia de cobre, pistas y componentes próximos que pudieran afectar la comunicación Wi-Fi.

5. DESARROLLO DEL FIRMWARE

El firmware fue desarrollado en el IDE de Arduino v2.3.10 para el microcontrolador ESP32-S3 y se organizó en módulos independientes, responsables de la adquisición de

señales, el cálculo del proceso, la generación de la salida, las interfaces de usuario y el registro de datos.

Las distintas funciones se ejecutan con períodos de actualización propios, permitiendo atender las tareas del sistema dentro de un mismo ciclo principal.

5.1. Organización general

El archivo principal contiene la configuración del hardware, el estado general del equipo y la coordinación de los módulos. Las funciones específicas se separaron en las siguientes bibliotecas:

Módulo	Función principal
Leer_4_20mA	Adquisición mediante el ADC, filtrado y conversión de la entrada
Generar_4_20mA	Cálculo del modelo, aplicación del retardo y control del DAC
MenuUsuario	Navegación y modificación de parámetros desde el equipo
Teclado y Display	Lectura de pulsadores y presentación de información en la pantalla OLED
InterfaceWiFi	Servidor web, comunicación con el navegador y configuración de red
SDLogger	Creación y escritura de archivos CSV en la tarjeta microSD
ProcessPresets	Definición de las configuraciones iniciales de proceso
BuzzerLib	Generación de avisos sonoros

Al iniciar el equipo se recuperan de la memoria no volátil los parámetros guardados, se inicializan los periféricos y se verifica la disponibilidad del DAC y de la tarjeta microSD. La salida se mantiene desactivada hasta que el usuario la habilita.

En el ciclo principal se atienden, en este orden general, los pulsadores, el buzzer, la entrada analógica, el modelo y la salida, el menú, la tarjeta microSD, la interfaz web, el display y el registro de datos. Cada módulo determina internamente si corresponde realizar una nueva actualización.

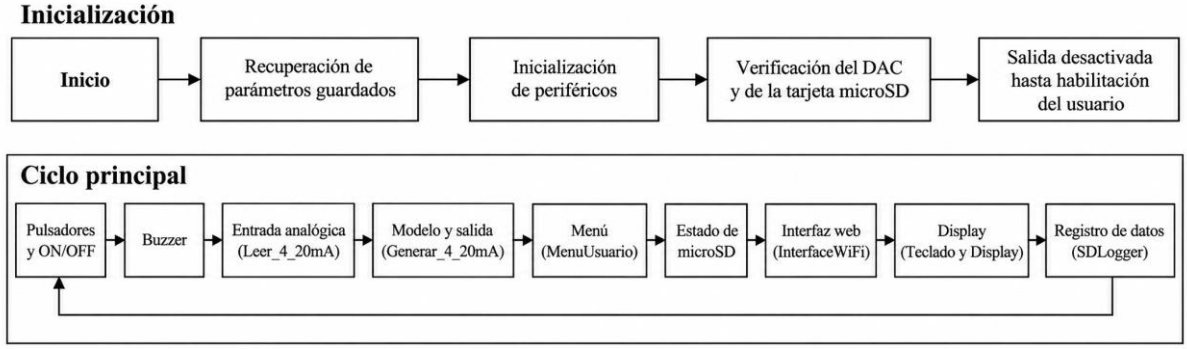


Figura 5.1 – Organización general del firmware

5.2. Adquisición y procesamiento de la entrada

La entrada se adquiere mediante el ADC de 12 bits del ESP32-S3. Cada 2 ms se realizan cuatro conversiones consecutivas y se calcula su promedio:

$$x[k] = \frac{x1 + x2 + x3 + x4}{4}$$

Este promedio reduce las variaciones entre conversiones individuales. A continuación, se aplica un filtro digital IIR de primer orden:

$$y[k] = y[k-1] + \frac{[x[k] - y[k-1]]}{8}$$

El filtro corrige en cada actualización un octavo de la diferencia entre la nueva lectura y el valor filtrado anterior. Su implementación utiliza aritmética de punto fijo con ocho bits fraccionarios.

El valor filtrado se convierte a corriente mediante una calibración de dos puntos:

$$I[k] = 4 + 16 \cdot \frac{[y[k] - RAW_4]}{[RAW_{20} - RAW_4]}$$

donde RAW4 y RAW20 son las cuentas obtenidas durante la calibración de 4 mA y 20 mA, respectivamente.

La entrada utilizada por el modelo se expresa como porcentaje:

$$Entrada[k] = 100 \cdot \frac{[y[k] - RAW_4]}{[RAW_{20} - RAW_4]}$$

Para el cálculo del proceso, este resultado se limita al intervalo de 0 a 100 %. La conversión a corriente se conserva sin limitar para detectar valores inferiores a 3,8 mA o superiores a 20,2 mA.

La calibración de entrada puede realizarse desde las interfaces del equipo. En cada punto se promedian 128 conversiones del ADC y se almacena el resultado en la memoria no volátil. El firmware verifica que RAW20 sea mayor que RAW4 y que exista una separación mínima de 300 cuentas.

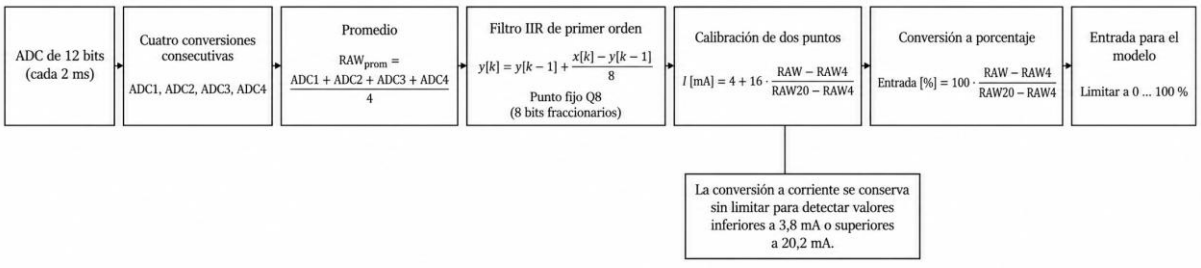


Figura 5.2 – Procesamiento digital de la entrada

5.3. Cálculo del proceso y generación de la salida

En modo automático, el modelo utiliza un período discreto de 50 ms y toma como entrada el último valor filtrado disponible. La referencia temporal se conserva entre ejecuciones, permitiendo mantener la correspondencia entre el tiempo real transcurrido y los pasos

discretos del modelo. Los coeficientes se calculan nuevamente cuando se modifica la ganancia K, la constante de tiempo τ o el período de actualización.

El retardo se implementa mediante un buffer circular que conserva las muestras anteriores de la entrada. La cantidad de posiciones utilizadas se obtiene mediante:

$$N = redondeo\left(\frac{\theta}{Ts}\right)$$

donde θ es el retardo configurado y Ts es el período de actualización del modelo. El buffer dispone de 601 posiciones, por lo que admite hasta 600 muestras de retardo. Con Ts=50ms, el valor máximo configurable de 30,0s queda representado por:

$$N = \frac{30}{0{,}05} = 600$$

En consecuencia, el retardo se representa en múltiplos de 50ms y puede ajustarse entre 0 y 30,0s. Después de calcular la salida del proceso, el firmware limita su valor al intervalo de 0 % a 100 % y aplica las alteraciones de medición configuradas. El offset y el ruido se incorporan según:

$$y_{m[k]} = y[k] + Offset + n[k]$$

donde n[k] representa una componente aleatoria de distribución uniforme centrada en cero. El resultado se limita al intervalo de 0 % a 100 %. Cuando se activa el congelamiento, el valor de salida se conserva hasta deshabilitar la función.

En modo manual no se calcula la dinámica del proceso ni el retardo. El usuario establece directamente el valor porcentual de la variable simulada, sobre el cual continúan actuando el offset, el ruido y el congelamiento.

Durante el funcionamiento automático, el valor manual sigue la salida interna del proceso antes de aplicar las modificaciones de medición. Al pasar a modo manual, este valor se utiliza como punto de partida y el offset, el ruido y el congelamiento se aplican una sola vez. Esto evita que el offset se duplique durante el cambio de modo. Si el ruido está habilitado, puede existir una pequeña variación aleatoria propia de la función configurada.

El resultado final se convierte a corriente mediante:

$$I_{out} = 4 + 16 \cdot \frac{Salida[\%]}{100}$$

La corriente se transforma en una cuenta del DAC utilizando los puntos de calibración de salida:

$$RAW = RAW_4 + \left[\frac{I_{out} - 4}{16}\right] \cdot (RAW_{20} - RAW_4)$$

El valor obtenido se limita al rango de 12 bits y se envía al MCP4725 mediante el bus I2C. Cuando la salida está habilitada, un valor de 0% corresponde a 4mA. Cuando se deshabilita, el firmware escribe directamente RAW=0 para llevar el circuito de salida a su estado de apagado.

La calibración de salida permite ajustar por separado las cuentas correspondientes a 4mA y 20mA. Durante este procedimiento el firmware puede mantener temporalmente uno de los extremos de la escala para facilitar la medición con un instrumento externo.

5.4. Interfaces y almacenamiento de la configuración

La interfaz local se implementa mediante una máquina de estados que diferencia la pantalla principal, la navegación por el menú, la edición de valores y la presentación de mensajes. Los pulsadores poseen un antirrebote de 25 ms. Las teclas de incremento y decremento también incorporan repetición automática para acelerar la modificación de valores.

La información presentada en el display se sincroniza cada 200 ms en la pantalla principal y cada 100 ms durante la navegación por los menús, la edición de parámetros y la presentación de mensajes. Las tendencias gráficas de entrada y salida se actualizan cada 250 ms y se almacenan temporalmente en memoria para representar la evolución reciente de ambas variables.

La interfaz web se ejecuta mediante un servidor HTTP alojado en el ESP32-S3. El navegador consulta periódicamente el estado del equipo en formato JSON y envía las modificaciones mediante solicitudes HTTP. Los cambios recibidos se aplican sobre la misma configuración utilizada por la interfaz local, por lo que ambas formas de operación actúan sobre un único estado interno.

Al habilitar Wi-Fi, el equipo intenta conectarse durante 12 segundos a la última red configurada. Si no puede hacerlo, crea el punto de acceso abierto “Simulador420_Wifi”, desde el cual puede accederse a la interfaz y configurar una nueva red.

Los parámetros del proceso, los ajustes de medición, los puntos de calibración y las preferencias de las interfaces se almacenan en la memoria no volátil del ESP32-S3. Esto permite recuperarlos después de una interrupción de la alimentación. Los estados operativos temporales, como la habilitación de la salida y la captura de datos, se inicializan nuevamente en cada arranque.



Figura 5.3 – Interfaz web del simulador

5.5. Registro de datos

El registro se implementa mediante una tarjeta microSD conectada por SPI. Al iniciar una captura, el firmware verifica la disponibilidad de la tarjeta y crea un archivo CSV con un nombre comprendido entre “HistoryData_01.csv” y “HistoryData_99.csv”.

Cuando la captura se inicia desde la interfaz web, el encabezado se genera según las columnas seleccionadas por el usuario. Cuando se inicia desde la interfaz local, se registran todas las columnas disponibles. El registro se realiza con un período nominal de un segundo. Cada muestra se agrega al archivo y, si se modifican las columnas durante una captura, se inicia un nuevo archivo con el encabezado correspondiente.

Si ocurre un error de escritura o se retira la tarjeta, la captura se detiene y el equipo informa la falla mediante sus interfaces. La página web permite consultar y descargar el último archivo generado.

Por defecto, el archivo contiene las siguientes columnas:

- Tiempo_ms
- Entrada_%
- Salida_%
- Salida_activa
- Modo
- Preset
- Ganancia_K
- Tau_s
- Retardo_s
- Ruido_%
- Offset_%
- Errores
- Salida_por_calibracion

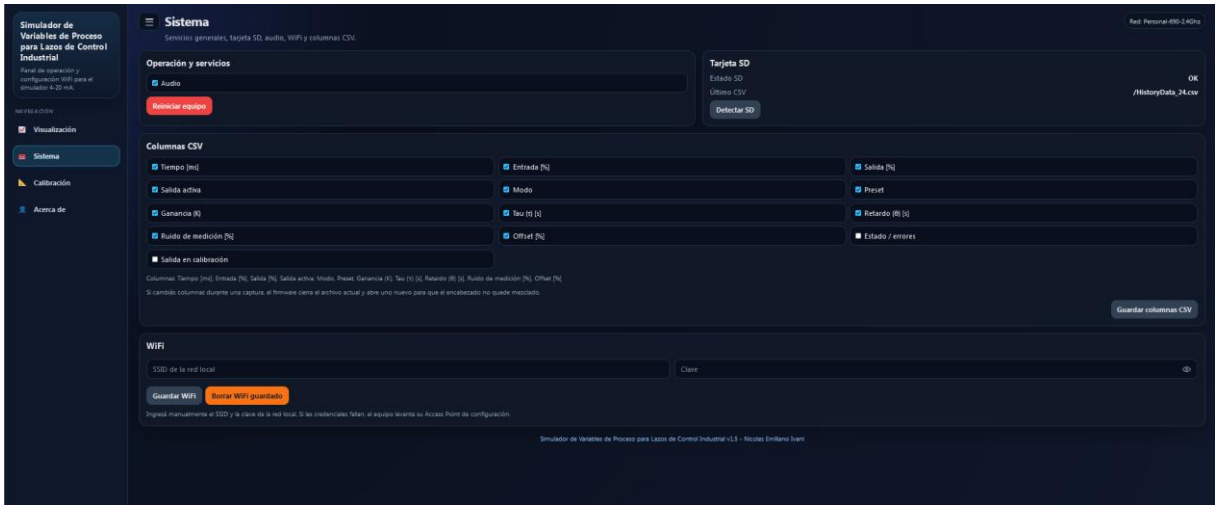


Figura 5.4 – Selección de columnas desde la web

	A
1	Tiempo_ms,Entrada_%,Salida_%,Salida_activa,Modo,Preset,Ganancia_K,Tau_s,Retardo_s,Ruido_%,Offset_%,Errores
2	30,0.17,47.59,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
3	1030,0.17,40.57,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
4	2030,0.20,34.59,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
5	3037,0.17,29.79,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
6	4037,0.13,25.41,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
7	5037,0.26,21.67,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
8	6053,1.42,18.68,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
9	7053,6.21,16.10,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
10	8053,13.36,14.53,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
11	9071,21.65,14.22,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
12	10071,29.22,15.16,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
13	11071,33.98,16.95,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
14	12087,35.60,19.22,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
15	13087,33.45,21.62,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
16	14087,28.46,23.42,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
17	15096,22.08,24.25,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
18	16096,15.50,24.07,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
19	17096,10.68,22.93,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
20	18111,8.10,21.32,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
21	19111,8.07,19.40,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
22	20111,10.74,17.73,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
23	21120,15.04,16.68,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
24	22120,20.23,16.35,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
25	23120,24.79,16.87,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"
26	24142,28.03,17.90,SI,"Automatico","Presion",1.000,5.000,0.500,0.00,0.00,"Sin errores"

Figura 5.5 – Ejemplo de registro generado

6. INTEGRACIÓN CON EL PLC

Para comprobar el funcionamiento del simulador se implementó un lazo de control utilizando un PLC Phoenix Contact ILC 191 ME/AN. El montaje permite utilizar la salida del controlador como entrada del modelo simulado y devolver la respuesta calculada al PLC como variable de proceso.

6.1. Equipamiento y conexión

El sistema de prueba está compuesto por los siguientes elementos:

- PLC Phoenix Contact ILC 191 ME/AN.
- Módulo de entrada analógica IB IL AI 4/I-PAC, código 2700458.
- Módulo de salida analógica IB IL AO 4/I/4-20-ECO, código 2702497.
- Fuente de alimentación de 24 Vcc.
- Simulador de Variables de Proceso para Lazos de Control Industrial.

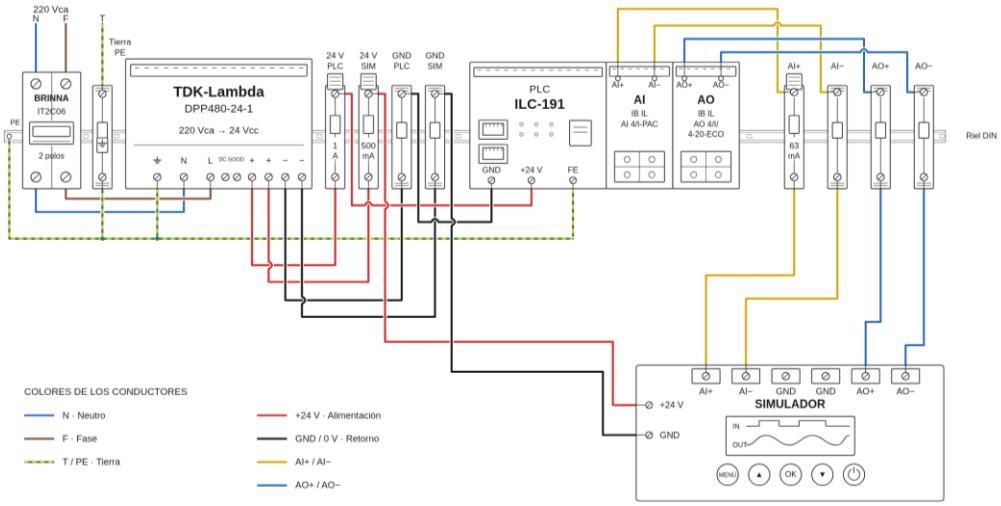


Figura 6.1 – Esquema de conexión y alimentación del PLC y el simulador.

El conexionado entre el PLC y el simulador se realiza mediante dos pares de conductores para las señales analógicas de 4–20 mA. Los terminales AO+ y AO− del módulo de salida analógica del PLC se conectan a los terminales AO+ y AO− del simulador, respectivamente, permitiendo transmitir la variable manipulada (MV). De igual manera, los terminales AI+ y AI− del módulo de entrada analógica del PLC se conectan a los terminales AI+ y AI− del simulador, para recibir la variable de proceso (PV) generada. En ambos casos se utiliza el primer canal de los módulos analógicos.

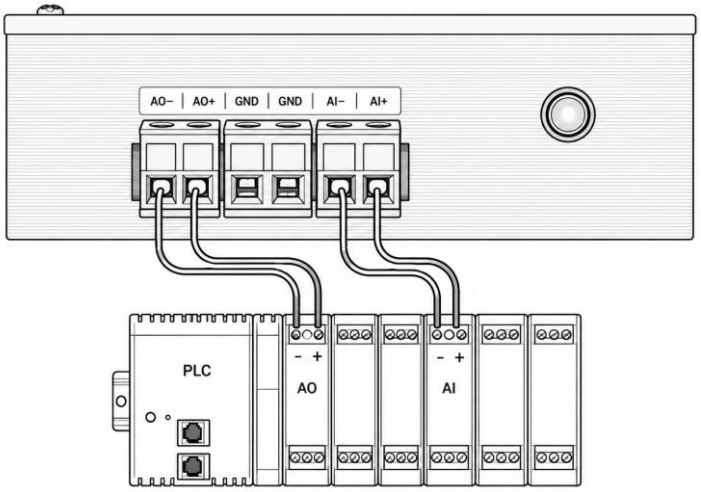


Figura 6.2 – Esquema de conexión entre los módulos analógicos del PLC y el simulador.

El PLC y el simulador se alimentan mediante una fuente externa de 24 Vcc, utilizando ramas de alimentación independientes.

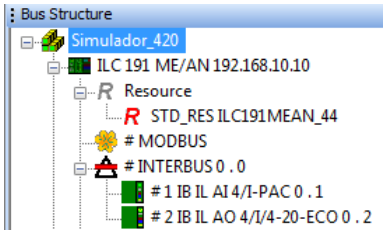


Figura 6.3 – Configuración del PLC y del módulo de entrada analógica

6.2. Programa de control

El proyecto fue desarrollado en PC Worx 6.30.1668. La señal proveniente del simulador se adquiere mediante el primer canal del módulo de entrada analógica y se convierte a porcentaje mediante una calibración de dos puntos.

Durante las pruebas se obtuvieron aproximadamente 700 cuentas para 4 mA y 3480 cuentas para 20 mA. El escalado utilizado se expresa como:

$$PV[\%] = 100 \cdot \frac{AI1_{Raw} - AI_{Raw_{4mA}}}{AI_{Raw_{20mA}} - AI_{Raw_{4mA}}}$$

El resultado se limita entre 0 % y 100 % y se utiliza como variable de proceso del controlador.

El control se implementa mediante un bloque FPID. Las variables utilizadas incluyen la consigna, la variable de proceso, la salida del controlador, los parámetros Kp, Ti y Td y la selección entre modo automático y manual.

El controlador se ejecuta dentro de una tarea de 100 ms. Su salida porcentual se convierte al rango utilizado por el módulo analógico, comprendido entre 0 y 16000 cuentas:

$$AO_{Raw} = 16000 \cdot \frac{MV[\%]}{100}$$

Este valor se envía al módulo de salida, que genera la señal de 4–20 mA utilizada como entrada del simulador.

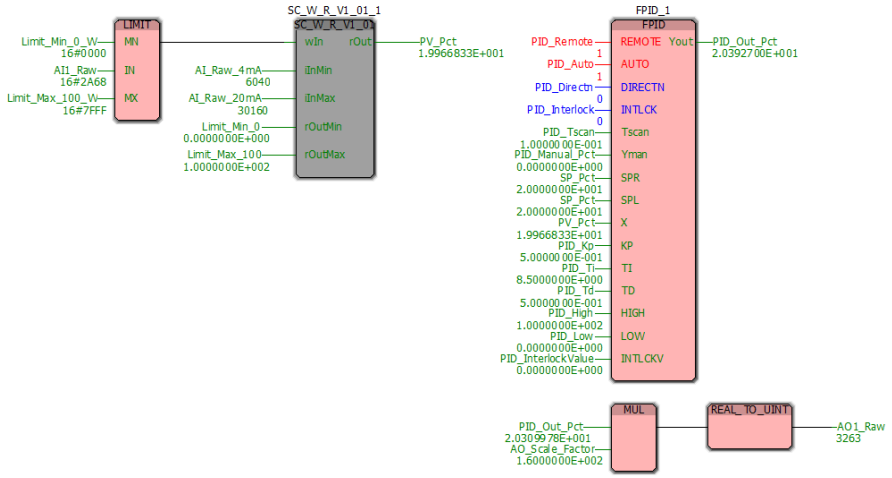


Figura 6.3 – Implementación del controlador FPID en PC Worx

6.3. Interfaz de operación

La operación del lazo se realiza mediante una pantalla desarrollada en WebVisit. La interfaz permite visualizar la variable de proceso PV, la consigna SP y la salida del controlador MV.

También permite modificar los parámetros Kp, Ti y Td, seleccionar el funcionamiento automático o manual y establecer el valor de salida cuando el controlador se encuentra en modo manual.

Un gráfico de tendencias representa la evolución de PV, SP y MV, permitiendo observar la respuesta del lazo durante los ensayos.

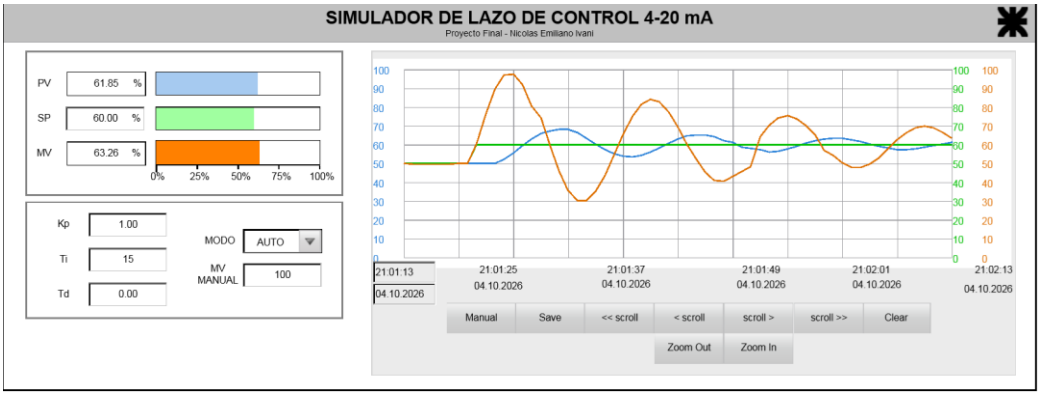


Figura 6.4 – Pantalla de operación del lazo en WebVisit

7. ENSAYOS Y RESULTADOS

La verificación del simulador se dividió en ensayos eléctricos, comprobaciones funcionales con el PLC, pruebas del modelo de proceso y ensayos sobre las alteraciones de medición.

Los ensayos permitieron comprobar el funcionamiento de las etapas analógicas de entrada y salida, la respuesta dinámica del modelo y la integración del simulador dentro de un lazo de control cerrado.

7.1. Verificación de las señales de entrada y salida

Entrada de 4–20mA

La entrada analógica se verificó aplicando distintos valores desde la salida del PLC y registrando el porcentaje indicado por el simulador, la cuenta del ADC y la corriente correspondiente. Se utilizaron once puntos distribuidos entre 0% y 100% de la escala.

AO PLC [%]	ENTRADA SIMULADOR [%]	CUENTA ADC	CORRIENTE [mA]
0%	0%	523	3,99
10%	10%	752	5,59
20%	20%	984	7,19
30%	30%	1219	8,78
40%	40%	1458	10,38
50%	50%	1701	11,98
60%	60%	1942	13,58
70%	70%	2182	15,18
80%	80%	2420	16,78
90%	90%	2662	18,38
100%	100%	2913	19,99

Tabla 7.1 – Verificación de la entrada de 4–20mA

Los resultados obtenidos muestran una correspondencia adecuada entre la señal aplicada por el PLC y el valor interpretado por el simulador en todo el rango de trabajo.

Salida de 4–20mA

La salida se verificó configurando distintos porcentajes en el simulador y registrando la corriente generada y el valor recibido por la entrada analógica del PLC.

SALIDA SIMULADA [%]	AI PLC [%]	CUENTA DAC	CORRIENTE [mA]
0%	0%	700	4,03
10%	10,11%	978	5,64
20%	20,19%	1256	7,25
30%	30,28%	1534	8,87
40%	40,26%	1812	10,46
50%	50,28%	2090	12,06
60%	60,23%	2368	13,66
70%	70,24%	2646	15,27
80%	80,19%	2924	16,85

90%	90,28%	3191	18,40
100%	100%	3480	20,02

Tabla 7.2 – Verificación de la salida de 4–20mA

Los valores medidos presentan una evolución prácticamente lineal entre 4mA y 20mA. La señal recibida por el PLC mantiene una buena correspondencia con el porcentaje configurado en el simulador, verificando el funcionamiento conjunto del DAC, el circuito de salida y la entrada analógica del controlador.

7.2. Pruebas funcionales e integración con el PLC

Una vez verificadas las señales analógicas, se realizó la prueba del sistema conectado al PLC Phoenix Contact ILC 191 ME/AN.

Se comprobó el funcionamiento completo del recorrido de las señales, la salida analógica del PLC se utilizó como entrada del simulador, el modelo calculó la respuesta del proceso y la señal generada regresó al PLC mediante el módulo de entrada analógica.

Posteriormente se operó el controlador FPID en lazo cerrado, utilizando la señal proveniente del simulador como variable de proceso. Se verificó que ante modificaciones de la consigna el controlador actuara sobre su salida y que el simulador generara la respuesta dinámica correspondiente, realimentando nuevamente al PLC.

También se comprobaron los modos automático y manual, la modificación de los parámetros del controlador desde WebVisit y la visualización de PV, SP y MV durante el funcionamiento.

Las pruebas permitieron verificar el funcionamiento conjunto del PLC, los módulos analógicos y el simulador, completando el lazo de control previsto para el banco de ensayo.

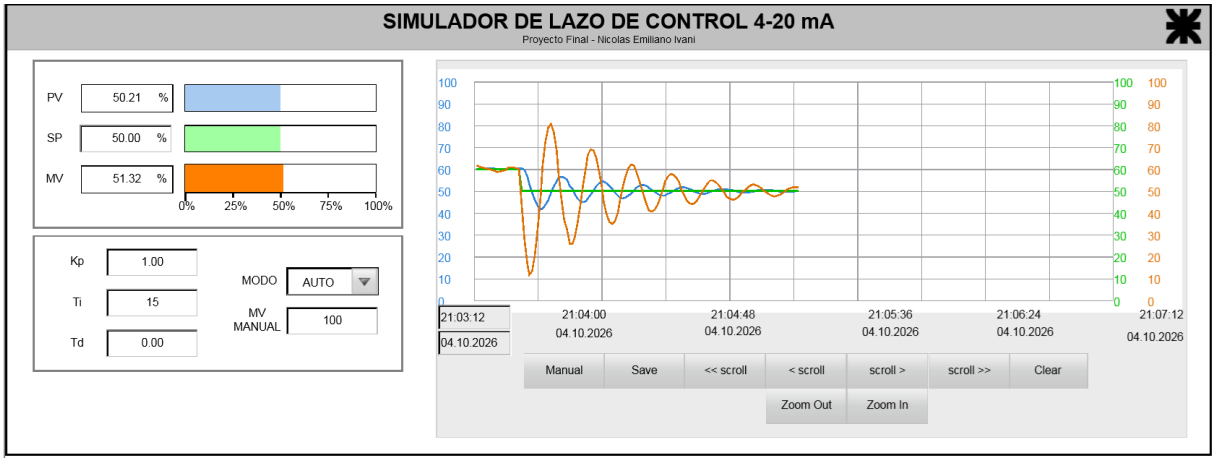


Figura 7.1 – Respuesta del lazo de control con el PLC.

7.3. Verificación del modelo de proceso

La respuesta del modelo se verificó mediante ensayos de escalón, registrando en la tarjeta microSD la entrada, la salida y los parámetros configurados.

La ganancia se determinó a partir de:

$$K_{medida} = \frac{\Delta y}{\Delta u}$$

La constante de tiempo se obtuvo identificando el instante en que la salida alcanzó el 63,2 % de su variación total:

$$y_{63,2} = y_0 + 0{,}632 \cdot (y_\infty - y_0)$$

El retardo se calculó como:

$$\theta_{medida} = t_{inicio} - t_{escalón}$$

Y la constante de tiempo:

$$\tau_{medida} = t_{63,2} - t_{inicio}$$

donde t_{inicio} corresponde al instante en que comienza a observarse la respuesta de la salida luego de aplicado el escalón.

Se realizaron tres ensayos, en los cuales se aplicó un escalón de entrada modificando la salida del PLC de 20 % a 60 %. Para cada prueba se utilizaron diferentes valores de constante de tiempo y retardo en el modelo de proceso del simulador, manteniendo en todos los casos una ganancia configurada K = 1.

Ensayo	K medida	τ config. [s]	τ medida [s]	θ config. [s]	θ medida [s]
1er Ensayo	0,999	10,0	10,01	0,0	0,0
2do Ensayo	0,997	10,0	10,00	2,0	2,19
3er Ensayo	0,999	2,0	2,00	30,0	30,41

Tabla 7.3. Resultados de la verificación del modelo de proceso.

Los resultados muestran una buena correspondencia entre los parámetros configurados y medidos. En los tres ensayos la ganancia permaneció próxima a la unidad y la constante de tiempo coincidió prácticamente con el valor configurado.



Figura 7.2 – Respuesta del modelo en el primer ensayo



Figura 7.3 – Respuesta del modelo en el segundo ensayo



Figura 7.4 – Respuesta del modelo en el tercer ensayo

7.4. Verificación de offset, ruido y congelamiento

Las funciones de offset, ruido de medición y congelamiento se verificaron con el simulador conectado al PLC y el controlador FPID operando en lazo cerrado. Para el ensayo se utilizaron los parámetros Kp=1, Ti=10s y Td=0,5s en el controlador, mientras que el modelo del simulador se configuró con K=1, τ=5s y θ=0,5s.

Inicialmente se estabilizó el lazo sin alteraciones. Al aplicar un offset de +10 puntos porcentuales, la PV aumentó de forma inmediata y el controlador redujo su salida para compensar el error introducido. Una vez alcanzado el nuevo equilibrio, la PV regresó a un valor próximo a la consigna, mientras que la MV se estabilizó alrededor del 40 %.

Posteriormente se aplicó un ruido de medición de ±5 puntos porcentuales. Durante esta condición la PV presentó variaciones entre 43,32 % y 55,87 %, provocando modificaciones continuas de la salida del controlador, que se mantuvo entre 34,24 % y 61,81 %.

Finalmente se activó el congelamiento de la medición. La PV permaneció fija en 49,82 % aun cuando la salida del controlador continuó modificándose. Ante un cambio de consigna realizado durante esta condición, la MV alcanzó el 100 % mientras la PV permaneció

congelada. Al liberar la función, la medición volvió a responder al estado del proceso y el lazo recuperó posteriormente su condición de operación.

Los resultados permitieron comprobar tanto la aplicación de las alteraciones sobre la señal simulada como la respuesta del controlador frente a mediciones desplazadas, ruidosas o congeladas.

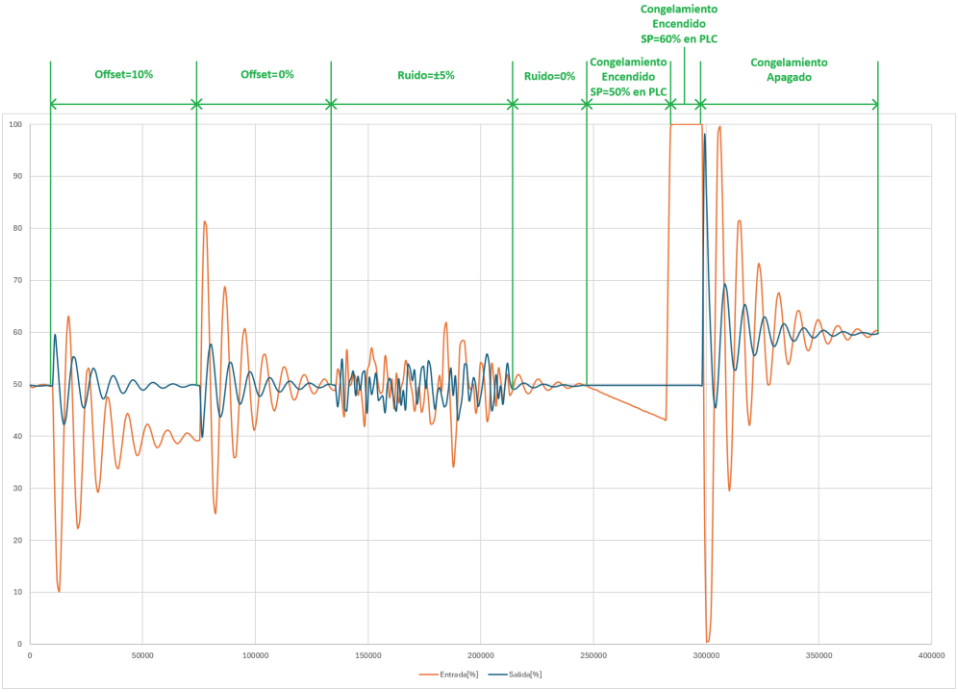


Figura 7.5 – Respuesta del lazo ante offset, ruido de medición y congelamiento

8. CONCLUSIONES

El desarrollo realizado permitió implementar un simulador capaz de integrarse a un lazo de control industrial mediante señales analógicas de 4–20 mA. El equipo adquiere la salida del controlador, calcula la respuesta de un modelo dinámico y genera una nueva señal que puede utilizarse como variable de proceso para realimentar el PLC.

El modelo de primer orden con retardo permite modificar directamente la ganancia K, la constante de tiempo τ y el retardo θ. Los ensayos realizados mostraron una buena correspondencia entre los parámetros configurados y la respuesta obtenida, incluyendo la verificación del retardo máximo de 30 s.

Las etapas analógicas de entrada y salida fueron verificadas en distintos puntos del rango de 4–20 mA. Posteriormente, el simulador se integró con un PLC Phoenix Contact ILC 191 ME/AN y se comprobó el funcionamiento completo del lazo mediante un controlador FPID.

También se verificaron las funciones de offset, ruido de medición y congelamiento. Estas pruebas permitieron observar no solo su aplicación sobre la variable simulada, sino también la respuesta del controlador frente a alteraciones de la medición.

La incorporación de una interfaz local, acceso mediante Wi-Fi y registro de datos en tarjeta microSD permite utilizar el equipo tanto para operación como para análisis posterior de los ensayos. La organización modular del firmware facilita además la modificación o incorporación de nuevas funciones.

En función de los resultados obtenidos, se considera cumplido el objetivo de desarrollar un prototipo funcional destinado a prácticas y experimentación con lazos de control industrial.

Trabajos futuros

Como posibles ampliaciones pueden evaluarse la incorporación de otros modelos de proceso, una mayor cantidad de canales analógicos y aislación galvánica de las señales. La estructura modular del firmware también permite continuar desarrollando nuevas funciones y estrategias de simulación a partir de la plataforma implementada.

9. Bibliografía

[1]. Espressif Systems. (s. f.). Preferences. En Arduino ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/preferences.html

[2]. Espressif Systems. (s. f.). Sleep modes. En ESP-IDF programming guide para ESP32-S3. https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/sleep_modes.html

[3]. Espressif Systems. (s. f.). Wi-Fi API. En Arduino ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

[4]. Espressif Systems. (2025). ESP32-S3 series datasheet (Versión 2.1) [Hoja de datos]. https://documentation.espressif.com/esp32-s3_datasheet_en.pdf

[5]. Espressif Systems. (2026). ESP32-S3-WROOM-1 & ESP32-S3-WROOM-1U datasheet (Versión 1.8) [Hoja de datos]. https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf

[6]. Fluke Corporation. (s. f.). Fluke 754 documenting process calibrator-HART [Información de producto]. https://www.fluke.com/en-us/product/calibration-tools/multifunction-calibrators/fluke-754

[7]. Fluke Corporation. (s. f.). Fluke 789 ProcessMeter [Información de producto]. https://www.fluke.com/en-us/product/calibration-tools/ma-loop-calibrators/fluke-789

[8]. The Foxboro Company. (1998). 762CNA SINGLE STATION MICRO controller (MI 018-885) [Manual de instrucciones].

[9]. The Foxboro Company. (1998). 762CSA SINGLE STATION MICRO controller: Shelf mounted (MI 018-889) [Manual de instrucciones].

[10]. Littelfuse. (2025). TVS diodes: Surface mount—600 W—SMBJ series (Versión 4) [Hoja de datos]. https://www.littelfuse.com/assetdocs/tvs-diodes-smbj-series-datasheet?assetguid=ba555e99-a12d-4f72-a0b6-86b06c67171e

[11]. Microchip Technology. (2009). MCP4725: 12-bit digital-to-analog converter with EEPROM memory in SOT-23-6 (DS22039D) [Hoja de datos]. https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ProductDocuments/DataSheets/22039d.pdf

[12]. Microchip Technology. (2020). MCP6001/1R/1U/2/4: 1 MHz low-power op amp (DS20001733L) [Hoja de datos]. https://ww1.microchip.com/downloads/aemDocuments/documents/MSLD/ProductDocuments/DataSheets/MCP6001-1R-1U-2-4-1-MHz-Low-Power-Op-Amp-DS20001733L.pdf

[13]. onsemi. (2024). BC327: PNP epitaxial silicon transistor (BC327-FCS/D, revisión 2) [Hoja de datos]. https://www.onsemi.com/download/data-sheet/pdf/bc327-d.pdf

[14]. onsemi. (2024). BC337: NPN epitaxial silicon transistor (BC337-FSC/D, revisión 3) [Hoja de datos]. https://www.onsemi.com/download/data-sheet/pdf/bc337-fsc-d.pdf

[15]. Phoenix Contact. (s. f.). IB IL AI 4/I-PAC: Analog module (Artículo 2700458) [Ficha técnica de producto]. https://www.phoenixcontact.com/en-pc/products/inline-terminal-ib-il-ai-4-i-pac-2700458

[16]. Phoenix Contact. (s. f.). IB IL AO 4/I/4-20-ECO: Analog module (Artículo 2702497) [Ficha técnica de producto]. https://www.phoenixcontact.com/en-de/products/analog-module-ib-il-ao-4-i-4-20-eco-2702497

[17]. Phoenix Contact. (s. f.). PC WORX BASIC LIC: Programming software (Artículo 2985275) [Información de producto]. https://www.phoenixcontact.com/en-us/products/programming-pc-worx-basic-lic-2985275

[18]. Phoenix Contact. (s. f.). WEBVISIT 6 PRO: Software de visualización (Artículo 2700949) [Información de producto]. https://www.phoenixcontact.com/es-es/productos/software-de-visualizacion-webvisit-6-pro-2700949

[19]. Phoenix Contact. (2023). Installing and operating the ILC 191 ME/AN and ILC 191 ME/INC Inline controllers (UM EN ILC 191 ME/X, revisión c05; documento 8607_en_c05) [Manual del usuario]. Archivo: um_en_ilc_191_me_x_with_atex_relevant_printing_8607_en_c05.pdf

[20]. Prats, F., & Sosa, A. (s. f.). Monitorización de señales de 4–20 mA de un controlador PID con Arduino y análisis gráfico con Matlab [Trabajo final de Electrónica Industrial no publicado].
Archivo: InformeProyectoFinal_ElectronicaIndustrial_Prats_Sosa.pdf

[21]. Real Games. (s. f.). Level control. En Factory I/O documentation. https://docs.factoryio.com/manual/scenes/level-control/

[22]. Siemens. (s. f.). SIMIT: Plataforma de simulación para la puesta en marcha virtual [Información de producto]. https://mall.industry.siemens.com/mall/es/WW/Catalog/Products/10165864?tree=CatalogTree

[23]. Sino Wealth. (s. f.). SH1106: 132 × 64 dot matrix OLED/PLED segment/common driver with controller (Versión 2.3) [Hoja de datos]. https://www.crystalfontz.com/controllers/SinoWealth/SH1106/

[24]. Texas Instruments. (2024). Industry-standard dual operational amplifiers (SLOS068AB, revisión AB; familia LM158/LM258/LM358/LM2904) [Hoja de datos]. https://www.ti.com/lit/ds/symlink/lm358.pdf

[25]. Paz, M. E., y Uribe, J. P. (2020). Equipo de Simulación de Escenario Industrial [Proyecto final de carrera de Ingeniería Electrónica, Universidad Tecnológica Nacional, Facultad Regional Bahía Blanca].

ANEXO A – FRAGMENTOS PRINCIPALES DEL FIRMWARE

A.1 Adquisición y filtrado de la entrada

Ubicación en el firmware: src\Leer_4_20mA\Leer_4_20mA.cpp

Función: Leer_4_20mA::update()

La adquisición de la entrada se realiza periódicamente mediante el ADC del ESP32-S3. En cada actualización se promedian varias conversiones y posteriormente se aplica un filtro IIR de primer orden.

```
_raw = sampleAveraged_();
const int32_t rawQ8 = static_cast<int32_t>(_raw) << 8;
if (!_hasSample) {
  _filteredQ8 = rawQ8;
  _hasSample = true;
} else if (_filterShift == 0) {
  _filteredQ8 = rawQ8;
} else {
  _filteredQ8 += (rawQ8 - _filteredQ8) >> _filterShift;
}
```

Fragmento A.1 – Adquisición y filtrado digital de la entrada

Con filterShift=3, la ecuación implementada corresponde a:

$$y[k] = y[k-1] + \frac{x[k] - y[k-1]}{8}$$

El promedio de las conversiones se realiza mediante la función auxiliar sampleAveraged_() de la misma biblioteca.

A.2 Modelo discreto y temporización

Ubicación en el firmware: src\Generar_4_20mA\Generar_4_20mA.cpp

Función: Generar_4_20mALib::updateFromInputmA().

El modelo se ejecuta con un período nominal de 50 ms. La referencia temporal se mantiene de forma acumulativa y, si alguna tarea retrasa momentáneamente el ciclo principal, se ejecutan los pasos discretos pendientes.

```
const uint32_t pendingSteps =
    elapsedMs / _cfg.sampleTimeMs;
_lastUpdateMs +=
    pendingSteps * _cfg.sampleTimeMs;
for (uint32_t i = 0; i < pendingSteps; ++i) {
```

```
    _lastDelayedInputPct =
      pushAndGetDelayedInput(_lastInputPct);
    _lastProcessPct =
      (_a * _lastProcessPct) +
      (_b * _lastDelayedInputPct);
  }
```

Fragmento A.2 – Ejecución temporal del modelo de proceso

La ecuación implementada es:

$$y[k] = a \cdot y[k-1] + b \cdot u[k-N]$$

con:

$$a = e^{-\frac{T_S}{\tau}}$$

$$b = K \cdot (1 - a)$$

Los coeficientes se calculan dentro de la misma biblioteca cuando se modifica la función de transferencia.

A.3 Implementación del retardo

Ubicación en el firmware: src\Generar_4_20mA\Generar_4_20mA.cpp

Función: Generar_4_20mALib::pushAndGetDelayedInput().

La capacidad máxima del buffer se define en Generar_4_20mA.h mediante MAX_DELAY_SAMPLES.

```
    _delayBuffer[_delayIndex] = inputPct;
    uint16_t readIndex = _delayIndex;
    if (_delaySamples > 0) {
      readIndex =
        (_delayIndex + MAX_DELAY_SAMPLES
         - _delaySamples) % MAX_DELAY_SAMPLES;
    }
    const float delayed =
      _delayBuffer[readIndex];
    _delayIndex++;
    if (_delayIndex >= MAX_DELAY_SAMPLES)
      _delayIndex = 0;
```

Fragmento A.3 – Buffer circular utilizado para el retardo

La cantidad de muestras se obtiene mediante:

$$N = round\left(\frac{\theta}{T_s}\right)$$

El buffer dispone de 601 posiciones, permitiendo representar hasta 600 muestras de retardo. Con T_s=50ms, esto corresponde a un retardo máximo de 30 s.

A.4 Alteraciones de la medición

Ubicación en el firmware: src\Generar_4_20mA\Generar_4_20mA.cpp

Función: Generar_4_20mALib::applySensorModel().

El offset y el ruido se aplican después del cálculo del proceso. El congelamiento conserva el primer valor obtenido al activar la función.

```
float sensorPct =
   processPct +
   _cfg.sensorOffsetPct +
   randomNoisePct();
if (_cfg.sensorFaultMode ==
   SENSOR_FAULT_FROZEN) {
 if (!_freezeLatched) {
   _frozenSensorPct = sensorPct;
   _freezeLatched = true;
 }
 sensorPct = _frozenSensorPct;
}
return clampf(sensorPct, 0.0f, 100.0f);
```

Fragmento A.4 – Aplicación de alteraciones sobre la medición

De forma simplificada:

$$y_{m[k]} = y[k] + Offset + n[k]$$

donde n[k] representa el ruido aleatorio configurado.

A.5 Apagado y modo de bajo consumo

Ubicación en el firmware: programa principal SimuladorIndustrial.ino, principalmente en las funciones shutdownPeripherals() y enterDeepSleep().

Durante el apagado, el programa deshabilita la salida analógica, finaliza la captura de datos, detiene Wi-Fi, tarjeta microSD, buzzer y pantalla, y posteriormente coloca al ESP32-S3 en modo Deep-sleep.

La configuración de la fuente de despertar y el ingreso al modo de bajo consumo se realiza mediante:

```
esp_sleep_enable_ext0_wakeup(
   static_cast<gpio_num_t>(PIN_PWR_BTN), 1);
esp_deep_sleep_start();
```

Fragmento A.5 – Ingreso del ESP32-S3 en modo Deep-sleep

El pulsador conectado al GPIO 15 queda configurado como fuente de despertar. Al accionarlo nuevamente, el microcontrolador reinicia la ejecución desde la secuencia de inicialización.

ANEXO B – GUÍA PARA FUTURAS MODIFICACIONES

El firmware se dividió en módulos independientes para facilitar el mantenimiento y la incorporación de nuevas funciones.

Archivo o módulo	Ubicación principal	Función
SimuladorIndustrial.ino	Programa principal	Inicialización, coordinación general, encendido y apagado
Leer_4_20mA.cpp/.h	Biblioteca Leer_4_20mA	Adquisición, filtrado, calibración y diagnóstico de entrada
Generar_4_20mA.cpp/.h	Biblioteca Generar_4_20mA	Modelo dinámico, retardo, alteraciones y salida DAC
ProcessPresets.h	Biblioteca ProcessPresets	Definición de presets y rangos del modelo
MenuUsuario.cpp/.h	Biblioteca MenuUsuario	Menú local y modificación de parámetros
Display.cpp/.h	Biblioteca Display	Pantalla OLED, tendencias e indicadores
InterfaceWiFi.cpp/.h	Biblioteca InterfaceWiFi	Interfaz web, configuración y servicios HTTP
SDLogger.cpp/.h	Biblioteca SDLogger	Registro CSV en microSD
Teclado.cpp/.h	Biblioteca Teclado	Lectura, antirrebote y repetición de pulsadores
BuzzerLib.cpp/.h	Biblioteca BuzzerLib	Avisos sonoros

Tabla B.1 – Organización del firmware

Los presets se encuentran definidos en ProcessPresets.h, dentro del arreglo PROCESS_PRESET_DEFINITIONS:

```
static const ProcessPresetDefinition
PROCESS_PRESET_DEFINITIONS[] = {
  {PROCESS_PRESET_LEVEL,
   "Nivel", "Nivel",
   1.00f, 10.0f, 0.8f},
  {PROCESS_PRESET_FLOW,
   "Caudal", "Caudal",
   1.00f, 2.0f, 0.2f},
  {PROCESS_PRESET_PRESSURE,
   "Presion", "Presion",
   1.00f, 5.0f, 0.5f},
  {PROCESS_PRESET_TEMPERATURE,
   "Temperatura", "Temp",
   1.00f, 20.0f, 2.0f}
};
```